

Anwendungsbeschreibung

PLC App

SPS-Laufzeitumgebung für ctrlX OS 03VRS

Schutzvermerk

© Bosch Rexroth AG 2025-08

Alle Rechte vorbehalten, auch bezüglich jeder Verfügung, Verwertung, Reproduktion, Bearbeitung, Weitergabe sowie für den Fall von Schutzrechtsanmeldungen.

Haftungsausschluss

Die angegebenen Daten dienen allein der Produktbeschreibung. Aufgrund stetiger Weiterentwicklung unserer Produkte kann eine Aussage über eine bestimmte Beschaffenheit oder eine Eignung für einen bestimmten Einsatzzweck aus unseren Angaben nicht abgeleitet werden. Die Angaben entbinden den Verwender nicht von eigenen Beurteilungen und Prüfungen. Es ist zu beachten, dass unsere Produkte einem natürlichen Verschleiß- und Alterungsprozess unterliegen.

DOK-XCORE*-PLC***V03**-AP03-DE-P

DC-AE/PAG-SW (MiNi)

Inhaltsverzeichnis

1	Über diese Dokumentation	5
2	Wichtige Gebrauchshinweise	5
2.1	Bestimmungsgemäßer Gebrauch	5
2.1.1	Einführung	5
2.1.2	Einsatz- und Anwendungsbereiche	6
2.2	Nicht bestimmungsgemäßer Gebrauch	7
3	Sicherheitshinweise	7
4	Einführung und Übersicht	7
4.1	PLC App – Grundlagen	7
4.2	PLC App – Grundlagen	10
5	Zielsysteme	12
6	Data Layer-Anbindung	13
7	Multicore-Funktionalität	14
7.1	Einführung	14
7.2	Lizenzierung der Multicore-Funktionalität	15
7.3	Lizenzierung der Multicore-Funktionalität	16
7.4	Multicore im IEC-Programm	16
7.4.1	Konfiguration der PLC-Tasks	16
7.5	Konfiguration der PLC-Tasks	19
8	Flexible Konfiguration des Speichers für remanente SPS-Daten	22
9	Anbindung des SPS-Laufzeitsystems an das ctrlX OS Systemverhalten	23
10	Optimierter Zugriff auf komplexe IEC-Datentypen über den Data Layer	24
10.1	Verwendung von Strukturen	25
10.2	Verwendung von Arrays von Strukturen	27
10.3	Verwendung von Strukturen mit Bits	29
10.4	Verwendung von Enums	33
10.5	Methodenaufrufe	36
10.6	Einstellungsmöglichkeiten	38
10.6.1	Enum-Darstellung	38
10.6.2	Browsen von Array-Variablen über OPC UA	40
10.7	Einschränkungen	40
11	Taskkonfiguration	40
11.1	Allgemeine Informationen zur Taskkonfiguration	40
11.2	Ereignisquellen für extern-ereignisgesteuerte SPS-Tasks	41
11.3	Unterstützung der Automatischen Programm-Konfiguration des ctrlX Schedulers	41
11.4	Frei konfigurierbare, externe Ereignisquellen	42
12	Erstellen von externen POEs	43
13	ctrlX Bedienoberfläche – Elemente	44
13.1	Navigation	44
13.1.1	Seitennavigation – PLC App	44
13.2	Fenster	45
13.2.1	Fenster – „PLC“	45

13.3	Header	46
13.3.1	Header – „PLC“	46
13.4	Registerkarten	47
13.4.1	Registerkarte – „Anwendungen“	47
13.4.2	Registerkarte – „Boot-Anwendungen“	48
13.4.3	Registerkarte – „Projektinformationen“	48
14	Weiterführende Dokumentationen	49
14.1	Übersicht	49
14.2	ctrlX AUTOMATION	49
14.3	ctrlX WORKS	50
14.4	ctrlX OS	50
14.5	ctrlX OS Apps	51
15	Service und Support	55
16	Index	57

1 Über diese Dokumentation

Ausgaben dieser Dokumentation

Ausgabe	Datum	Bemerkung
01	2024 – 09	Erstausgabe zur PLC App Version PLC 3.2
02	2025 – 02	Ausgabe zur PLC App Version PLC 3.4 Überarbeitete Inhalte: ➔ PLC App – Grundlagen ➔ Registerkarte – „Anwendungen“
03	2025 – 08	Ausgabe zur PLC App Version PLC 3.6 Überarbeitete Inhalte: ➔ Multicore im IEC-Programm ➔ Optimierter Zugriff auf komplexe IEC-Datentypen über den Data Layer Neue Inhalte: ➔ Taskkonfiguration

2 Wichtige Gebrauchshinweise

2.1 Bestimmungsgemäßer Gebrauch

2.1.1 Einführung

Produkte von Rexroth werden nach dem jeweiligen Stand der Technik entwickelt und gefertigt.

Vor ihrer Auslieferung werden die Produkte auf ihren betriebssicheren Zustand hin überprüft.

 **WARNUNG**

Personen- und Sachschäden durch falschen Gebrauch der Produkte!

Die Produkte dürfen nur bestimmungsgemäß eingesetzt werden.

Wenn die Produkte nicht bestimmungsgemäß eingesetzt werden, dann können Situationen entstehen, die Sach- und Personenbeschädigung nach sich ziehen.

Wichtige Gebrauchshinweise

ACHTUNG
Schäden bei nicht bestimmungsgemäßigem Gebrauch

Für Schäden bei nicht bestimmungsgemäßigem Gebrauch der Produkte leistet Rexroth als Hersteller keinerlei Gewährleistung, Haftung oder Schadensersatz. Die Risiken bei nicht bestimmungsgemäßigem Gebrauch der Produkte liegen allein beim Anwender.

Bevor Sie die Produkte der Firma Rexroth einsetzen, müssen die folgenden Voraussetzungen erfüllt sein, um einen bestimmungsgemäßen Gebrauch der Produkte zu gewährleisten:

- Jeder, der in irgendeiner Weise mit Rexroth Produkten umgeht, muss die entsprechenden Sicherheitsvorschriften und den bestimmungsgemäßen Gebrauch lesen und verstehen
- Sofern es sich bei den Produkten um Hardware handelt, müssen die Produkte in ihrem Originalzustand belassen werden; d. h. es dürfen keine baulichen Veränderungen an den Produkten vorgenommen werden. Softwareprodukte dürfen nicht dekompiert werden und ihre Quellcodes dürfen nicht verändert werden
- Beschädigte oder fehlerhafte Produkte dürfen nicht eingebaut oder in Betrieb genommen werden
- Es muss gewährleistet sein, dass die Produkte entsprechend den in der Dokumentation genannten Vorschriften installiert sind

2.1.2 Einsatz- und Anwendungsbereiche

Produkte der ctrlX Baureihe sind für Motion-/Logic-Anwendungen geeignet.

ACHTUNG

Produkte der ctrlX Baureihe dürfen nur mit den in dieser Dokumentation angegebenen Zubehör- und Anbauteilen benutzt werden. Nicht ausdrücklich genannte Komponenten dürfen weder angebaut noch angeschlossen werden. Gleiches gilt für Kabel und Leitungen.

Der Betrieb darf nur in den ausdrücklich angegebenen Konfigurationen und Kombinationen der Hardware-Komponenten und mit der in den jeweiligen Dokumentationen und den Funktionsbeschreibungen angegebenen und spezifizierten Soft- und Firmware erfolgen.

Produkte der ctrlX Baureihe sind für den Einsatz in ein- und mehrachsigen Antriebs- und Steuerungsaufgaben geeignet. Für den applikationsspezifischen Einsatz des Systems stehen Gerätetypen mit unterschiedlicher Ausstattung und unterschiedlichen Schnittstellen zur Verfügung.

Typische Anwendungsbereiche:

- Gebäudeautomatisierung
- IoT und Security Gateway bzw. Device
- Handling & Robotic

Steuerungen der ctrlX CORE Baureihe dürfen nur unter den in den weiterführenden Dokumentationen angegebenen Montage- und Installationsbedingungen, in der angegebenen Gebrauchslage und unter den angegebenen Umweltbedingungen (Temperatur, Schutzart, Feuchte, EMV u. a.) betrieben werden.

2.2 Nicht bestimmungsgemäßer Gebrauch

Die Verwendung von ctrlX-Produkten außerhalb der vorgenannten Anwendungsgebiete oder unter anderen als den in der Dokumentation beschriebenen Betriebsbedingungen und angegebenen technischen Daten gilt als "nicht bestimmungsgemäß".

ctrlX-Produkte dürfen nicht eingesetzt werden, wenn sie den folgenden Bedingungen ausgesetzt sind:

- Betriebsbedingungen, die die vorgeschriebenen Umgebungsbedingungen nicht erfüllen. Untersagt sind z. B. der Betrieb unter Wasser, unter extremen Temperaturschwankungen oder extremen Maximaltemperaturen
- Bei Anwendungen, die von Rexroth nicht ausdrücklich freigegeben sind

3 Sicherheitshinweise

Die Sicherheitshinweise, soweit in der vorliegenden Anwendungsdokumentation vorhanden, beinhalten bestimmte Signalwörter ("Gefahr", "Warnung", "Vorsicht", "Hinweis") und ggf. eine Signalgrafik (nach ANSI Z535.6-2006).

Das Signalwort soll die Aufmerksamkeit auf den Sicherheitshinweis lenken und bezeichnet die Schwere der Gefährdung.

Die Signalgrafik (Warndreieck mit Ausrufezeichen), welche den Signalwörtern "Gefahr", "Warnung" und "Vorsicht" vorangestellt wird, weist auf Gefährdungen für Personen hin.

Die Sicherheitshinweise in dieser Dokumentation werden wie folgt dargestellt:

 GEFAHR	Bei Nichtbeachtung dieses Sicherheitshinweises werden Tod oder schwere Körperverletzung eintreten.
 WARNUNG	Bei Nichtbeachtung dieses Sicherheitshinweises können Tod oder schwere Körperverletzung eintreten.
 VORSICHT	Bei Nichtbeachtung dieses Sicherheitshinweises können mittelschwere oder leichte Körperverletzung eintreten.
ACHTUNG	Bei Nichtbeachtung dieses Sicherheitshinweises können Sachschäden eintreten.

4 Einführung und Übersicht

4.1 PLC App – Grundlagen

Durch die Installation der „PLC App“ wird der Funktionsumfang der ctrlX CORE-Steuerung um eine SPS-Laufzeitumgebung ergänzt

Die Laufzeitumgebung basiert auf "CODESYS Control V3" und ist vollständig in die ctrlX OS-Systemumgebung integriert.

Die SPS-Programmierung und das Debugging erfolgt über das PC-basierte Engineering-Tool ctrlX PLC Engineering.

PLC App Lizenzierung

Der Betrieb der PLC App ist lizenzpflichtig.

Das Lizenzmodell enthält insgesamt vier Lizenzen, mit denen der Funktionsumfang je nach Bedarf skaliert werden kann.

Zum Betrieb der PLC App ist die Lizenz "PLC Basic" erforderlich.

Aufbauend auf der PLC Basic- Lizenz kann der Funktionsumfang durch Ergänzung von Upgrade-Lizenzen erweitert werden.

Lizenzen / Funktionsumfang

Lizenzen	PLC-Tasks	CPU-Kerne	E/A-Daten
PLC Basic	1	1	Bis zu 256 Byte
PLC Standard	Nicht limitiert	1	Bis zu 1024 Byte
PLC Advanced	Nicht limitiert	Nicht limitiert	Bis zu 2048 Byte
PLC Performance	Nicht limitiert	Nicht limitiert	Nicht limitiert

Lizenzen / Bestellinformationen

Lizenz	Beschreibung
PLC Basic	Basislizenz zum Betrieb der PLC App auf dem ctrlX-Gerät Typbezeichnung: SWL-XC*-PLC-PLC*XOS*****-BANN Materialnummer: R911421866
PLC Standard	Upgrade der PLC Basic Lizenz Erfordert folgende Lizenz auf dem ctrlX-Gerät: • PLC Basic Typbezeichnung: SWL-XC*-PLC-PLC*XOS*****-NNNN Materialnummer: R911421867
PLC Advanced	Upgrade der PLC Standard Lizenz Erfordert folgende Lizenzen auf dem ctrlX-Gerät: • PLC Basic • PLC Standard Typbezeichnung: SWL-XC*-PLC-PLC*XOS*****-ADNN Materialnummer: R911421868
PLC Performance	Upgrade der PLC Advanced Lizenz Erfordert folgende Lizenzen auf dem ctrlX-Gerät: • PLC Basic • PLC Standard • PLC Advanced Typbezeichnung: SWL-XC*-PLC-PLC*XOS*****-PRNN Materialnummer: R911421869

Automatische Lizenzprüfung

Bei jedem Laden einer SPS-Applikation (Bootprojekt, Download, Konfiguration laden) führt die SPS automatisch folgende Schritte zur Lizenzprüfung aus:

1. ➤ Auslesen der Lizenzen, die auf dem ctrlX-Gerät vorhanden sind (Basic, Standard, Advanced, Performance)
2. ➤ Erfassung der Applikationseigenschaften und Abgleich mit den vorhandenen Lizenzen

Prüfpunkte:

- Anzahl der Tasks: Single / Multi Task
- Anzahl der verwendeten CPU-Kerne
- Anzahl der verwendeten E/A's in den Projektknoten:
 - Realtime_Data (ethercat_master_instances_ethercatmaster)
 - CODESYS ProfiNet Controller / Device
 - CODESYS EtherNET/IP Scanner / Adapter
- ➔ Bei korrekter Lizenzierung kann das Projekt in die SPS geladen und in den Betriebszustand "RUN" geschaltet werden.
- ➔ Reaktionen bei nicht ausreichender Lizenzierung:
 - Der Download einer Applikation wird abgewiesen.
 - Die Umschaltung auf Betriebszustand "RUN" wird verweigert.
 - Fehlermeldung im ctrlX OS Diagnosesystem.
 - Fehlermeldung im plc Logger.
- ➔ Reaktion wenn Lizenzen während der Laufzeit ablaufen:
 - Funktionen der abgelaufenen Lizenz werden gestoppt

Erforderliche Lizenzen im PLC-Projekt ermitteln

ctrlX PLC Engineering bietet folgende Funktionen, um erforderliche Lizenzen zu ermitteln:

- **Erforderliche Lizenzen im Projekt ermitteln**
Die Registerkarte „Lizensierte Softwaremetriken“ im Geräteeditor des Steuerungsknotens zeigt die zur Lizenzierung zu berücksichtigenden Parameter im Projekt an.
- **Gegenüberstellung der vorhandenen und benötigten Lizenzen**
Bei verbundener Steuerung werden in der Registerkarte „Status
→ Lizenzen“ alle vorhandenen und alle erforderlichen Lizenzen aufgelistet.

Lizenzprüfung bei virtuellen ctrlX-Geräten

Für virtuelle ctrlX CORE Geräte gilt eine lizenzfreie Laufzeitbeschränkung von 4 Stunden. Nach Ablauf der Laufzeit wird die Steuerung automatisch heruntergefahren, kann aber erneut gestartet und für weitere 4-Stunden betrieben werden. Installierte Add-Ons der Firma Codesys definieren je nach individueller Konfiguration einen eigenen Zeitraum.

Weiterführende Informationen

Dokumentation

- ➔ [ctrlX OS - Apps Grundlagen](#)
- ➔ [ctrlX OS - Lizenzübersicht](#)

Bezugsquellen

- ➔ [ctrlX STORE](#)

Web-Links

- [↗ ctrlX AUTOMATION - Community](#)
- [↗ ctrlX AUTOMATION - How-to](#)
- [↗ ctrlX AUTOMATION - Forum](#)

4.2 PLC App – Grundlagen

Durch die Installation der „PLC App“ wird der Funktionsumfang der ctrlX CORE-Steuerung um eine SPS-Laufzeitumgebung ergänzt

Die Laufzeitumgebung basiert auf "CODESYS Control V3" und ist vollständig in die ctrlX CORE-Systemumgebung integriert.

Die SPS-Programmierung und das Debugging erfolgt über das PC-basierte Engineering-Tool ctrlX PLC Engineering.

PLC App Lizenzierung

Der Betrieb der PLC App ist lizenzpflichtig.

Das Lizenzmodell enthält insgesamt drei Lizenzen, mit denen der Funktionsumfang je nach Bedarf skaliert werden kann.

Zum Betrieb der PLC App ist die "Basic Single License" erforderlich.

Aufbauend auf der "Basic Single License" kann der Funktionsumfang durch Ergänzung von "In-App" Lizenzen erweitert werden.

Lizenz Bezeichnung	Beschreibung	Typ / Materialnummer
ctrlX PLC Basic Single license	Basislizenz zum Betrieb der PLC App • Betrieb einer SPS Task auf dem Zielgerät	SWL-XC*-PLC-PLC*****-BANN R911397817
ctrlX PLC Standard Single license	Ergänzende In-App Lizenz <u>Funktionserweiterung:</u> • Betrieb mehrerer SPS-Tasks auf dem Zielgerät möglich <u>Erfordert:</u> • ctrlX PLC Basic Single license	SWL-XC*-PLC-PLC*****-NNNN R911411765
ctrlX PLC Advanced Single license	Ergänzende In-App Lizenz <u>Funktionserweiterung:</u> • Betrieb mehrerer SPS-Tasks auf dem Zielgerät möglich • Verteilung der Tasks auf mehrere CPU-Kerne möglich <u>Erfordert:</u> • ctrlX PLC Basic Single license • ctrlX PLC Basic Standard Single license	SWL-XC*-PLC-PLC*****-ADNN R911401461

Automatische Lizenzprüfung

Bei jedem Laden einer SPS-Applikation (Bootprojekt, Download, Konfiguration laden) führt die SPS automatisch folgende Schritte zur Lizenzprüfung aus:

1. ➤ Auslesen der Lizenzen und Überprüfung, welche SPS-Lizenzen auf dem ctrlX-Gerät vorhanden sind (Basic, Standard, Advanced, Performance)
2. ➤ Prüfung der Applikationseigenschaften (Single Task, Multi Task, Multi Core) und Abgleich mit den vorhandenen Lizenzen
 - ➔ Bei korrekter Lizenzierung kann die SPS in den Betriebszustand "RUN" geschaltet werden
 - ➔ Reaktionen bei nicht ausreichender Lizenzierung:
 - Die Umschaltung auf Betriebszustand "RUN" wird verweigert
 - Fehlermeldung im ctrlX-CORE Diagnosesystem
 - Fehlermeldung im plc Logger
 - ➔ Reaktion wenn Lizenzen während der Laufzeit ablaufen:
 - Funktionen der abgelaufenen Lizenz werden gestoppt

Ausnahmen bei der automatischen Lizenzprüfung

Eine Ausnahme bei fehlender Lizenzierung bildet die "ctrlX PLC Advanced Single" Lizenz, die den Start der Applikation auch bei fehlender Lizenz erlaubt. In diesem Fall wird die Applikation jedoch automatisch nach Ablauf von 30 Minuten gestoppt.

Lizenzprüfung bei virtuellen ctrlX-Geräten

Für virtuelle ctrlX CORE Geräte gilt eine lizenzfreie Laufzeitbeschränkung von 4 Stunden. Nach Ablauf der Laufzeit wird die Steuerung automatisch heruntergefahren, kann aber erneut gestartet und für weitere 4-Stunden betrieben werden. Installierte Add-Ons der Firma Codesys definieren je nach individueller Konfiguration einen eigenen Zeitraum.

Berechtigungskonzept

Das Berechtigungskonzept für die PLC App sieht die folgenden Berechtigungen für den Zugriff auf SPS relevante Daten vor:

Berechtigung	ctrlX OS Web-Oberfläche	ctrlX PLC Engineering
SPS ansehen (PLC view)	• Lesender Zugriff auf SPS relevante Daten (Data Layer Knoten).	Verbindung zur Steuerung nicht möglich.
SPS bedienen (PLC operate)	• Lesender Zugriff auf SPS relevante Daten (Data Layer Knoten). • Eingeschränkter Schreibzugriff (Start/Stop/Reset warm/Reset cold).	Verbindung zur Steuerung nicht möglich.
SPS verwalten (PLC manage)	• Keine Einschränkungen.	Keine Einschränkungen.

Die Einstellungen können für den jeweiligen Benutzer in der Kategorie PLC vergeben werden, siehe: ➔ [ctrlX OS Web-Dokumentation](#)

Bitte berücksichtigen Sie bei der Vergabe von Benutzerechten auch die weiteren Kategorien.

Weiterführende Informationen**Dokumentation**

- [↗ ctrlX CORE - Apps Grundlagen](#)
- [↗ ctrlX CORE - Lizenzübersicht](#)

Bezugsquellen

- [↗ ctrlX App Store](#)

Web-Links

- [↗ ctrlX CORE - Community](#)
- [↗ ctrlX CORE - How-to](#)
- [↗ ctrlX CORE - Forum](#)

5 Zielsysteme

Die PLC App unterstützt Zielplattformen auf Basis ARM64 und x64.

Die Programmierung in ctrlX PLC Engineering erfolgt prozessorabhängig über die zugehörigen Geräte:

- ctrlX CORE ARM64
- ctrlX CORE x64

Die Abbildung der ctrlX CORE Hardwarevarianten auf das jeweilige Gerät ist in der folgenden Tabelle definiert:

ctrlX CORE Hardwarevariante	Auszuwählendes Gerät in ctrlX PLC Engineering
ctrlX CORE X2	ctrlX CORE ARM64
ctrlX CORE X3	ctrlX CORE ARM64
ctrlX CORE X3+	ctrlX CORE ARM64
ctrlX DRIVE +	ctrlX CORE ARM64
ctrlX CORE x7	ctrlX CORE x64
ctrlX CORE virtual	ctrlX CORE x64
ctrlX CORE OS	ctrlX CORE x64

Projektvorlagen

Beim Anlegen eines neuen Projekts in ctrlX PLC Engineering werden plattform-spezifische Projektvorlagen zur Auswahl bereitgestellt:

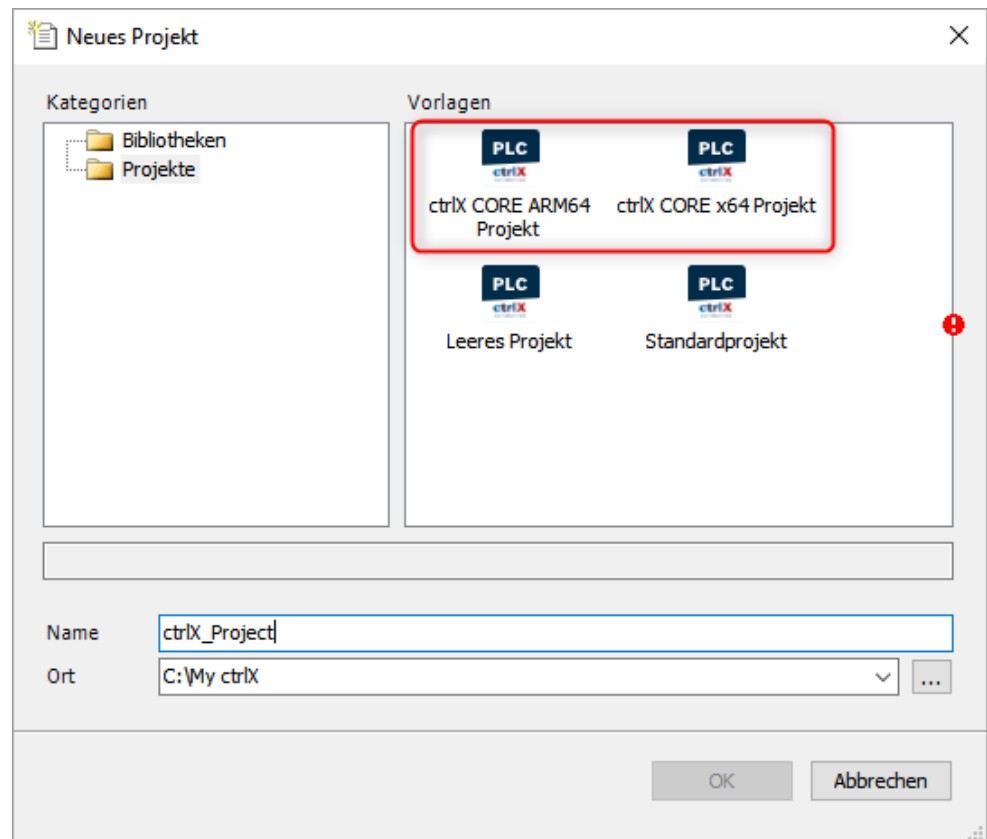


Abb. 1: Dialog zur Auswahl einer Projektvorlage in ctrlX PLC Engineering

Zykluszeiten / Speichergrößen / Bootprojekt

Bei allen Gerätevarianten beträgt die minimal einstellbare Taskzykluszeit 1ms. Die Größe des nutzbaren Retain-Speichers ist hardwareabhängig. Der Defaultwert ist 64 kB, kann aber nach unten reduziert bzw. je nach Hardwaregrenzen nach oben erweitert werden.

Die Ablage des Bootprojekts auf dem Zielgerät erfolgt in spezifischen Ordnern in der Konfiguration (App data).

6 Data Layer-Anbindung

Die PLC App veröffentlicht ihre relevanten Daten bzw. Kommandos über den Data Layer-Knoten „plc“. Die Unterknoten referenzieren allgemeine, sowie applikationsspezifische Daten und Kommandos.

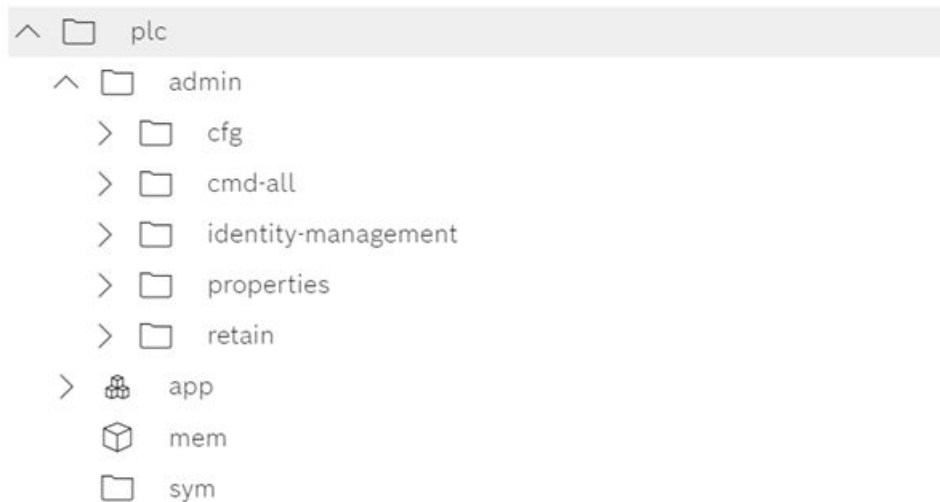


Abb. 2: Knoten "plc" im Data Layer der ctrlX OS Web-Oberfläche

7 Multicore-Funktionalität

7.1 Einführung

Die Multicore-Funktion ermöglicht die Nutzung mehrerer CPU-Kerne des ctrlX-Geräts zur parallelen Abarbeitung von SPS IEC-Tasks.

Vorteile:

- Gesteigerte Ausführungsgeschwindigkeit durch parallele Abarbeitung bei gleichzeitiger Ausnutzung der verfügbaren Prozessorleistung.
- Komplexe SPS-Logiken können auf verschiedene, unabhängige Taskgruppen und CPU-Kerne aufgeteilt werden.
- Trennung von Logikapplikation, Kommunikation und Zusatzfunktionen wie z. B. Motion oder Visualisierung.

Voraussetzungen

- ctrlX-Gerätetyp ab CORE X3 oder ctrlX DRIVE plus mit integrierter ctrlX CORE
- ctrlX-Version ab PLC-V-0110 oder höher
- Erforderliche Lizenzen auf dem ctrlX-Gerät:
 - PLC Basic
 - PLC Standard
 - PLC Advanced

Automatische Prüfung auf Multicore-Fähigkeit

Beim Hochlauf des ctrlX-Geräts werden die verfügbaren Prozessorkerne und deren Eigenschaft automatisch ermittelt (Realtime bzw. Non-Realtime). Das SPS-Laufzeitsystem fragt diese Eigenschaften beim Hochlauf ab und weist die System-Tasks dem ersten, verfügbaren Realtime-Prozessorkern zu. Die Ausführung der SPS-Tasks wird ebenfalls dem ersten, verfügbaren Realtime-Prozessorkern zugeordnet.

Über die Taskkonfiguration in ctrlX PLC Engineering kann der Anwender die Ausführung der SPS-Tasks auch anderen, verfügbaren Prozessorkernen manuell zuweisen. Für die Ausführung müssen die erforderlichen Lizenzen auf dem ctrlX-Gerät installiert sein, siehe:

➔ [Lizenzierung der Multicore-Funktionalität](#)



In ctrlX PLC Engineering werden in der Task-Konfiguration immer 4 Prozessorkerne für die Zuordnung angeboten, unabhängig von den tatsächlich vorhandenen Eigenschaften des Zielsystems. Wenn eine Applikation auf ein Zielsystem geladen wird, deren Prozessor-Konfiguration auf dem Zielgerät nicht ausgeführt werden kann, dann wird der Download abgebrochen und eine Fehlermeldung ausgegeben.

verwandte Themen:

- ➔ Apps und Lizenzen im ctrlX OS STORE
- ➔ Lizenz-Center "Quick Start Guide"
- ➔ Multicore und Taskkonfiguration - Grundlagen und Konfiguration

7.2 Lizenzierung der Multicore-Funktionalität

verwandte Themen:

- ➔ Lizenz-Center "Quick Start Guide"
- ➔ Apps und Lizenzen im ctrlX OS STORE

Die Verwendung der Multicore-Funktionalität auf dem ctrlX-Gerät erfordert folgende Lizenzen:

Lizenz	Beschreibung	Lizenz Typ-schlüssel	Material-nummer
PLC Basic	Basislizenz zum Betrieb der PLC App und des SPS-Laufzeitsystems auf dem ctrlX-Gerät	SWL-XC*-PLC-PLC*XOS*****-BANN	R911421866
PLC Standard	Upgrade der PLC Basic Lizenz	SWL-XC*-PLC-PLC*XOS*****-NNNN	R911421867
PLC Advanced	Upgrade der PLC Basic Lizenz	SWL-XC*-PLC-PLC*XOS*****-ADNN	R911421868

Die Upgrade-Lizenzen werden über die Web-Oberfläche der ctrlX CORE Steuerung installiert, siehe ➔ [weiterführende Dokumentation](#)

Multicore Projekt - Verhalten ohne ausreichende Lizenz

Bei unzureichender Lizenzierung werden SPS-Projekte, die für Multicore-Systeme konfiguriert sind, nur für 120 Minuten im Demo-Modus betrieben. Die Abarbeitung erfolgt in diesem Fall auf nur einem Prozessorkern.



Nach Ablauf der 120 Minuten geht die SPS mit einer Exception in den STOP-Zustand und im SPS-Logger wird folgende Meldung ausgegeben: „Multi-core demo mode expired! Reboot of the controller is necessary now!“

Bestehende Projekte auf Multicore umstellen

Wenn ein bestehendes Projekt, auf Multicore-Funktionalität umgestellt werden soll, ist zu beachten, dass die Steuerungs-Gerätebeschreibungsdatei (Device-Description) Version V1.10.0.1 oder höher ist.

Bei Projekten, die mit einer PLC Engineering-Version vor WRK-V-0110 erstellt wurden, ist es erforderlich, dass die Gerätebeschreibungsdatei der Steuerung aktualisiert wird, siehe Befehl ➔ [Gerät aktualisieren](#).

7.3 Lizenzierung der Multicore-Funktionalität

verwandte Themen:

- ➔ [Lizenz-Center "Quick Start Guide"](#)
- ➔ [Apps und Lizenzen im ctrlX OS STORE](#)

Die Verwendung der Multicore-Funktionalität auf der ctrlX CORE-Steuerung erfordert folgende Lizenz:

Bezeichnung	Beschreibung	Lizenz Typ-schlüssel	Material-nummer
PLC-Advance Lizenz	In-App Lizenz, zur Aktivierung der PLC Multicore Funktionalität auf der ctrlX Steuerung.	SWL-XC*-PLC-PLC*****_ADNN	R911401461

Die PLC-Advance Lizenz wird über die Web-Oberfläche der ctrlX CORE Steuerung installiert, siehe ➔ [weiterführende Dokumentation](#)

Multicore Projekt - Verhalten ohne PLC-Advance Lizenz

Wenn keine PLC-Advance Lizenz auf der ctrlX CORE Steuerung installiert ist, werden SPS Projekte, die für Multicore Systeme konfiguriert sind, für 120 Minuten im Demo-Modus betrieben. Die Abarbeitung erfolgt in diesem Fall auf einem Prozessorkern.



Nach Ablauf der 120 Minuten geht die SPS mit einer Exception in den STOP-Zustand und im SPS-Logger wird folgende Meldung ausgegeben: „Multi-core demo mode expired! Reboot of the controller is necessary now!“

Bestehende Projekte auf Multicore umstellen

Wenn ein bestehendes Projekt, auf Multicore-Funktionalität umgestellt werden soll, ist zu beachten, dass die Steuerungs-Gerätebeschreibungsdatei (Device-Description) Version V1.10.0.1 oder höher ist.

Bei Projekten, die mit einer PLC Engineering-Version vor WRK-V-0110 erstellt wurden, ist es erforderlich, dass die Gerätebeschreibungsdatei der Steuerung aktualisiert wird, siehe Befehl ➔ [Gerät aktualisieren](#).

7.4 Multicore im IEC-Programm

Die Multicore-Funktionalität kann zu Leistungssteigerungen bei der Abarbeitung des SPS-Programms beitragen, indem die IEC-Tasks und deren Last auf verschiedene CPU-Kerne aufgeteilt werden können.

Bei Projekten ohne Multicore-Funktionalität, erfolgt die komplette Abarbeitung aller Tasks auf CPU-Kern # 2.

- Anwender-IEC-Tasks
- Kommunikations-Tasks
- System-Tasks

7.4.1 Konfiguration der PLC-Tasks

Alle Tasks, die im Kontext der PLC-App laufen, werden auf dem ersten Echtzeit-Core der ctrlX OS Steuerung abgearbeitet. Über die Multicore-Option kann der Anwender die im SPS-Programm konfigurierten IEC Tasks konfigurieren und auf die vorhandenen CPU-Kerne verteilen. In ctrlX PLC Engineering stehen hierzu die Taskgruppen zur Verfügung, über welche die verfügbaren Cores zugeordnet werden:



Ab Version PLC-V-0206 können auch die PLC-System-Tasks (Kommunikation,...) den verschiedenen Cores zugeordnet werden.

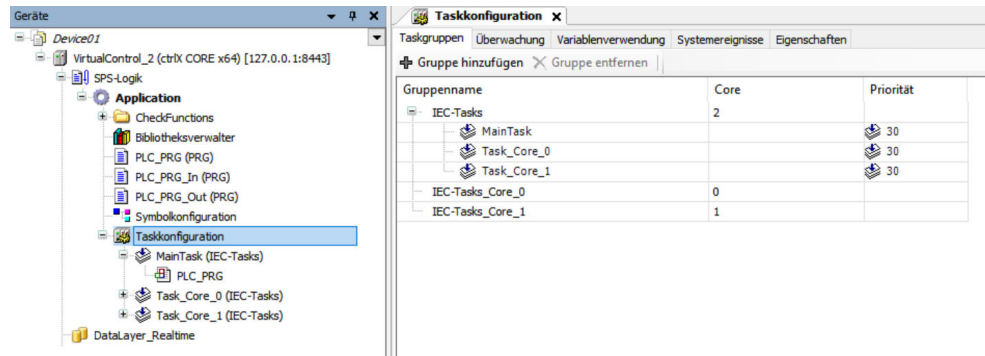


Abb. 3: ctrlX PLC Engineering Taskkonfiguration

Voraussetzungen

- Die ctrlX CORE Steuerung unterstützt die Multicore-Funktionalität (ab Device-Description V1.10.0.1)
- Sie haben in Ihrer Applikation mindestens zwei Tasks definiert, beispielsweise „MainTask (IEC-Task)“ und „LowTask (IEC-Task)“

Zuweisung konfigurieren

Die Zuweisung der Tasks auf die CPU-Kerne erfolgt über ctrlX PLC Engineering in der Registerkarte Taskgruppen, siehe [weiterführende Dokumentation](#).

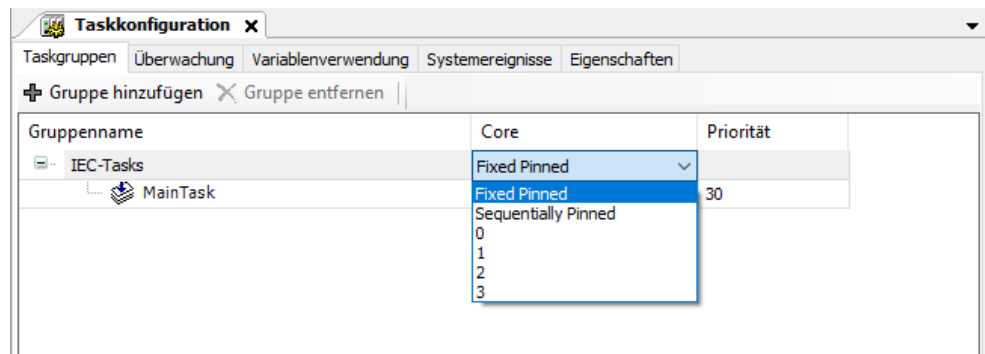


Abb. 4: Registerkarte Taskgruppen

Im Auslieferungszustand ist bereits die Taskgruppe "IEC-Tasks" in der Einstellung "Fixed Pinned" vorkonfiguriert, in der alle Tasks im CPU-Kern #2 abgearbeitet werden.

Um Tasks auf verschiedene Kerne aufzuteilen, erstellen Sie zuerst eine weitere Taskgruppe. Im Anschluss wählen Sie im Feld "Core" die CPU-Nummer, die für die Abarbeitung der zweiten Taskgruppe zuständig sein soll. Ziehen Sie jetzt die gewünschten Tasks per Drag & Drop in die neue Taskgruppe.

In der Einstellung "Sequentially pinned" werden die CPU-Kerne fortlaufend belegt



Bitte beachten Sie, dass auf den CPU-Kernen 0 und 1 das Betriebssystem ctrlX OS läuft.

Sollten Taskgruppen auf diesen Kernen laufen, sollte immer der Watchdog der IEC-Task aktiviert sein.

Funktionsbeschreibung

Zur Unterteilung der Funktionalität definiert das System vier (System-)Taskgruppen:

- **System**
Alle Tasks des Systems, wie Scheduler, Exception Handler etc.
- **Communication**
Alle Tasks der Kommunikation.
- **Visualization**
Alle Tasks die Visualisierung bearbeiten.
- **Asynchronous Operations**
Alle Tasks der asynchronen Operationen z.b: SysFileAsync

Die Abbildung dieser Taskgruppen und deren Konfiguration erfolgt im Data Layer am Knoten: plc->admin->runtime-settings->SysCpuMultiCore‘

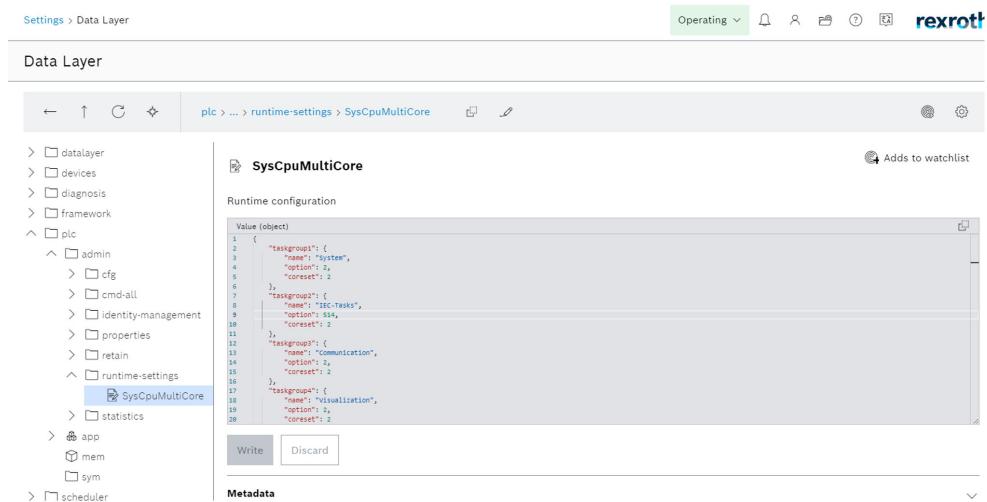


Abb. 5: ctrlX OS – Data Layer

Der Name und die Optionen der jeweiligen Taskgruppe sind fest und können nicht verändert werden. Über den Parameter “coreset” kann der Prozessorkern, auf dem die Abarbeitung dieser Taskgruppe erfolgen soll, ausgewählt werden. Die auf der ctrlX OS Steuerung für diese Zuordnung verfügbaren Prozessorkerne sind ebenfalls unter diesem Data Layer-Knoten abgebildet:

SysCpuMultiCore

Runtime configuration

Value (object)	
22	"taskgroup5": {
23	"name": "Asynchronous Operations",
24	"option": 2,
25	"coreset": 2
26	},
27	"ieccoreset": [
28	0,
29	1,
30	2,
31	3
32	]
33	}
34	

Abb. 6: ctrlX OS – Data Layer / verfügbare Prozessorkerne



IEC-Tasks sind immer höherprior gegenüber dem Betriebssystem und können durch eine zu hohe Auslastung das Betriebssystem stark verlangsamen oder unbedienbar machen.

Hinweise

- Änderungen der Standardkonfiguration wirken sich auf das Laufzeitverhalten des Gesamtsystems aus und sind nur für Experten empfohlen.
- Damit die neuen Einstellungen für die Taskgruppe(n) wirksam werden, muss die ctrlX OS Steuerung neu gestartet werden.

Weiterführende Informationen zur Multicore-Anwendung finden Sie in der PLC Engineering Dokumentation, siehe [↗ Multicore und Taskkonfiguration](#)

7.5 Konfiguration der PLC-Tasks

Alle Tasks, die im Kontext der PLC-App laufen, werden auf dem ersten Echtzeit-Core der ctrlX OS Steuerung abgearbeitet. Über die Multicore-Option kann der Anwender die im SPS-Programm konfigurierten IEC Tasks konfigurieren und auf die vorhandenen CPU-Kerne verteilen. In ctrlX PLC Engineering stehen hierzu die Taskgruppen zur Verfügung, über welche die verfügbaren Cores zugeordnet werden:



Ab Version PLC-V-0206 können auch die PLC-System-Tasks (Kommunikation,...) den verschiedenen Cores zugeordnet werden.

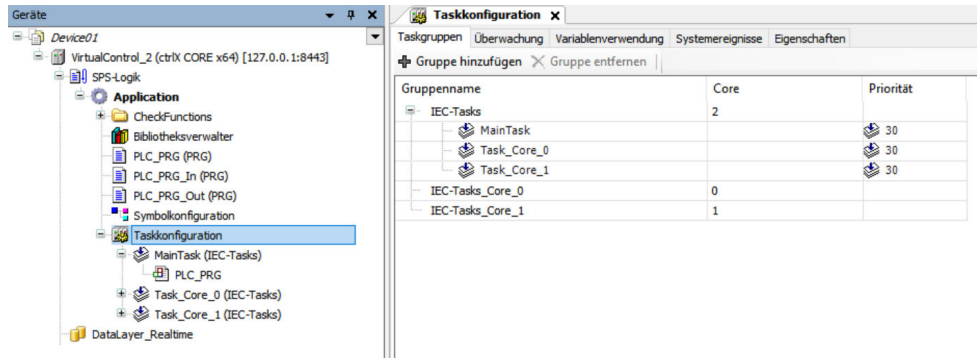


Abb. 7: ctrlX PLC Engineering Taskkonfiguration

Voraussetzungen

- Die ctrlX CORE Steuerung unterstützt die Multicore-Funktionalität (ab Device-Description V1.10.0.1)
- Sie haben in Ihrer Applikation mindestens zwei Tasks definiert, beispielsweise „MainTask (IEC-Task)“ und „LowTask (IEC-Task)“

Zuweisung konfigurieren

Die Zuweisung der Tasks auf die CPU-Kerne erfolgt über ctrlX PLC Engineering in der Registerkarte Taskgruppen, siehe [weiterführende Dokumentation](#).

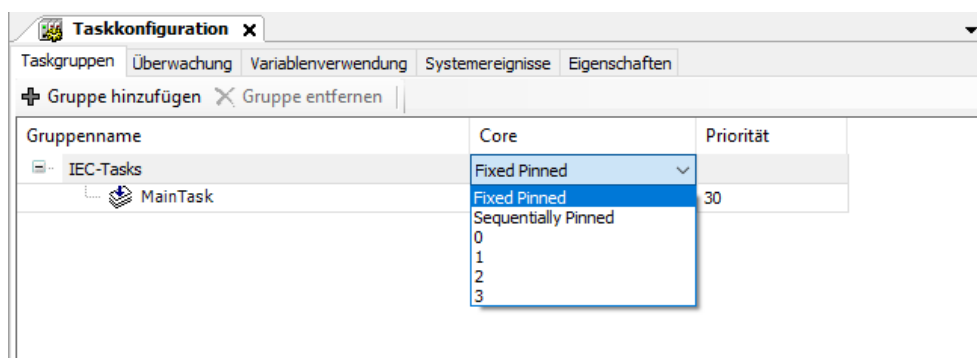


Abb. 8: Registerkarte Taskgruppen

Im Auslieferungszustand ist bereits die Taskgruppe "IEC-Tasks" in der Einstellung "Fixed Pinned" vorkonfiguriert, in der alle Tasks im CPU-Kern #2 abgearbeitet werden.

Um Tasks auf verschiedene Kerne aufzuteilen, erstellen Sie zuerst eine weitere Taskgruppe. Im Anschluss wählen Sie im Feld "Core" die CPU-Nummer, die für die Abarbeitung der zweiten Taskgruppe zuständig sein soll. Ziehen Sie jetzt die gewünschten Tasks per Drag & Drop in die neue Taskgruppe.

In der Einstellung "Sequentially pinned" werden die CPU-Kerne fortlaufend belegt



Bitte beachten Sie, dass auf den CPU-Kernen 0 und 1 das Betriebssystem ctrlX OS läuft.

Sollten Taskgruppen auf diesen Kernen laufen, sollte immer der Watchdog der IEC-Task aktiviert sein.

Funktionsbeschreibung

Zur Unterteilung der Funktionalität definiert das System vier (System-)Taskgruppen:

- **System**
Alle Tasks des Systems, wie Scheduler, Exception Handler etc.
- **Communication**
Alle Tasks der Kommunikation.
- **Visualization**
Alle Tasks die Visualisierung bearbeiten.
- **Asynchronous Operations**
Alle Tasks der asynchronen Operationen z.b: SysFileAsync

Die Abbildung dieser Taskgruppen und deren Konfiguration erfolgt im Data Layer am Knoten: plc->admin->runtime-settings->SysCpuMultiCore‘

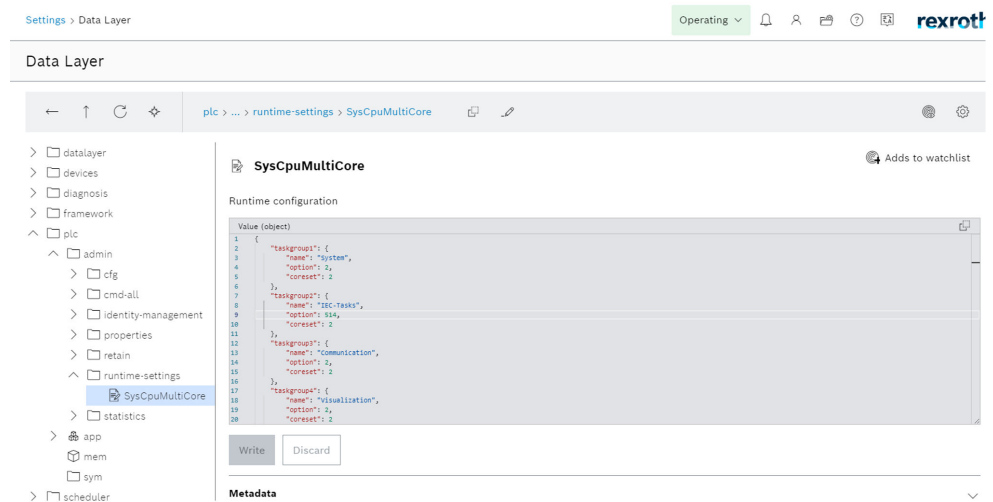


Abb. 9: ctrlX OS – Data Layer

Der Name und die Optionen der jeweiligen Taskgruppe sind fest und können nicht verändert werden. Über den Parameter “coreset“ kann der Prozessor-kern, auf dem die Abarbeitung dieser Taskgruppe erfolgen soll, ausgewählt werden. Die auf der ctrlX OS Steuerung für diese Zuordnung verfügbaren Prozessorkerne sind ebenfalls unter diesem Data Layer-Knoten abgebildet:

SysCpuMultiCore

Runtime configuration



```

Value (object)
22      "taskgroup5": {
23          "name": "Asynchronous Operations",
24          "option": 2,
25          "coreset": 2
26      },
27      "ieccoreset": [
28          0,
29          1,
30          2,
31          3
32      ]
33  }
34

```

Abb. 10: ctrlX OS – Data Layer / verfügbare Prozessorkerne



IEC-Tasks sind immer höherprior gegenüber dem Betriebssystem und können durch eine zu hohe Auslastung das Betriebssystem stark verlangsamen oder unbedienbar machen.

Hinweise

- Änderungen der Standardkonfiguration wirken sich auf das Laufzeitverhalten des Gesamtsystems aus und sind nur für Experten empfohlen.
- Damit die neuen Einstellungen für die Taskgruppe(n) wirksam werden, muss die ctrlX OS Steuerung neu gestartet werden.

Weiterführende Informationen zur Multicore-Anwendung finden Sie in der PLC Engineering Dokumentation, siehe [Multicore und Taskkonfiguration](#)

8 Flexible Konfiguration des Speichers für remanente SPS-Daten



Der Speicher für remanente SPS-Daten steht als Pool für alle SPS Applikationen zur Verfügung, die zur Laufzeit auf der Steuerung betrieben werden.

Remanente Variablen weisen die Eigenschaft auf, ihre Werte über einen Steuerungs-Ausschaltvorgang hinaus zu behalten.

Physikalisch stehen auf der ctrlX CORE-Steuerung bis zu 128 kB nichtflüchtiger Speicher zur Verfügung. Der Speicherplatz kann von den Apps angefordert und für remanente Variablen genutzt werden. Für die PLC App und die SPS-Applikationen stehen als Defaultwert 64 kB für Variablen vom Typ VAR RETAIN und VAR PERSISTENT zur Verfügung.

Ab der PLC App Version PLC-V-0108 kann die reservierte Speichergröße dynamisch auf bis zu 120 kB eingestellt werden.



Bei einer Änderung der Speichergröße ist ein Reboot der Steuerung erforderlich.

Bestehende Retain- und Retain Persistent Daten gehen verloren.

Speichergröße konfigurieren

1. Öffnen Sie die ctrlX OS Web-Oberfläche und navigieren Sie zum Fenster „PLC“, siehe [Dokumentation](#)
2. Klicken Sie im Header auf
 - ➔ Der Dialog „SPS-Retain-Speicher zuweisen (Bytes)“ öffnet sich
3. Geben Sie die gewünschte Speichergröße in Bytes ein.
Der mögliche Maximalwert ist hardwareabhängig. Wenn der Eingabewert außerhalb der minimalen bzw. maximalen Speichergröße liegt, wird der Minimal- bzw. Maximalwert eingetragen. Wenn Sie den Vorgang abbrechen bleibt der zuletzt gültige Wert erhalten.
4. Bestätigen Sie den Dialog
 - ➔ Die Steuerung führt einen Reboot durch

Die Änderung der Speichergröße ist damit abgeschlossen

9 Anbindung des SPS-Laufzeitsystems an das ctrlX OS Systemverhalten

Anbindung an den ctrlX OS System Handler

Die Verwaltung der SPS-Applikationen und der zugehörigen Tasks wird über den Scheduler des SPS-Laufzeitsystems übernommen.

Um auch das allgemeine Systemverhalten der ctrlX CORE Steuerung sicherzustellen, ist zusätzlich eine Anbindung an den ctrlX OS Systemstatus erfolgt. Diese bildet sich für die PLC App wie folgt ab:

- Die SPS-Applikationen werden im ctrlX OS System Status wie folgt berücksichtigt:
 - Jede SPS-Applikation wird beim System Handler der ctrlX OS Runtime registriert.
 - Der Systemstatus (Data Layer Knoten system->state) zeigt die aktuell auf das Gerät geladenen SPS-Applikationen an.
 - Die über die SPS-Applikation generierten IEC- Tasks werden in der Task-Übersicht des ctrlX OS Schedulers erfasst, die Task-Eigenschaften können jedoch nicht im zugehörigen Dialog geändert werden.
- Beim Umschalten des Betriebszustands der ctrlX CORE Steuerung werden die SPS-Applikationen wie folgt behandelt:
 - **Operating:** Alle auf dem Gerät vorhandenen SPS-Applikationen werden in den Zustand "RUN" versetzt.
 - **Setup:** Alle aktuell auf dem Gerät laufenden SPS-Applikationen werden in den Zustand "STOP" geschaltet.
 - **Service:** Alle aktuell auf dem Gerät laufenden SPS-Applikationen werden in den Zustand "STOP" geschaltet.

- Schalten des Status einer PLC-Applikation:
 - **Operating:** Jede SPS-Applikation kann beliebig von RUN nach STOP und umgekehrt schalten.
 - **Setup:** Alle auf dem ctrlX-Gerät konfigurierten SPS-Applikationen befinden sich zunächst im Zustand **STOP** und können in den Zustand RUN geschaltet werden. Sobald mindestens eine SPS-Applikation gestartet wurde, wechselt der Scheduler-Status nach "Partial Operating".
 - **Service:** Alle auf das Gerät geladenen SPS-Applikationen befinden sich im Zustand "STOP" und können nicht nach "RUN" geschaltet werden.

Zuordnung der PLC Tasks in Abhängigkeit von der Steuerungsarchitektur

Die Standardarchitektur für ctrlX CORE Steuerungen ist auf 4 Prozessorkerne (core0,...core3) ausgelegt. Dabei werden zwischen Realtime- und Non-Realtime-Prozessorkernen unterschieden. Für die Echtzeitanwendungen sind die Realtimekerne core 2 und core 3 reserviert, während auf core 0 und 1 die Standardanwendungen und das Linux Betriebssystem ausgeführt werden. Für die Ausführung der SPS Laufzeitumgebung und der SPS Applikationen ist der erste Echtzeitkern (core 2) reserviert.

Liegen nun Steuerungsarchitekturen vor, die von der Standardarchitektur abweichen, ermittelt der System Handler die auf der Hardware verfügbaren Prozessorkerne und weist diesen die Eigenschaften Realtime- oder Non-Realtime zu. Dem SPS Laufzeitsystem wird dabei immer der erste Realtimekern zugewiesen.



In ctrlX PLC Engineering werden unabhängig von der Zielsystemarchitektur über die Taskkonfiguration immer 4 Kerne für die Zuordnung von Taskgruppen angeboten. Wenn auf der Zielarchitektur die konfigurierten Kerne nicht vorhanden sind, wird der Downloadvorgang mit entsprechenden Hinweisen abgebrochen.

10 Optimierter Zugriff auf komplexe IEC-Datentypen über den Data Layer

Es existieren mehrere Möglichkeiten auf Variablen eines SPS-Programms im Data Layer zuzugreifen. Allerdings wurden Strukturen und Arrays von Strukturen bislang nur mir der originalen Struktur im Data Layer abgebildet. Mit Version PLC-V-0204 wurde eine neue Funktion eingeführt, die es ermöglicht, dass in der Symbolkonfiguration neue Datentypen für ausgewählte Variablen im Data Layer angelegt werden können.

Voraussetzung

Um die neue Funktion nutzen zu können müssen in ctrlX PLC Engineering folgende Optionen in den Einstellungen der Symbolkonfiguration aktiviert sein:

- Support OPC UA features
- Support calls to Functions, FBs, Methods and Programs

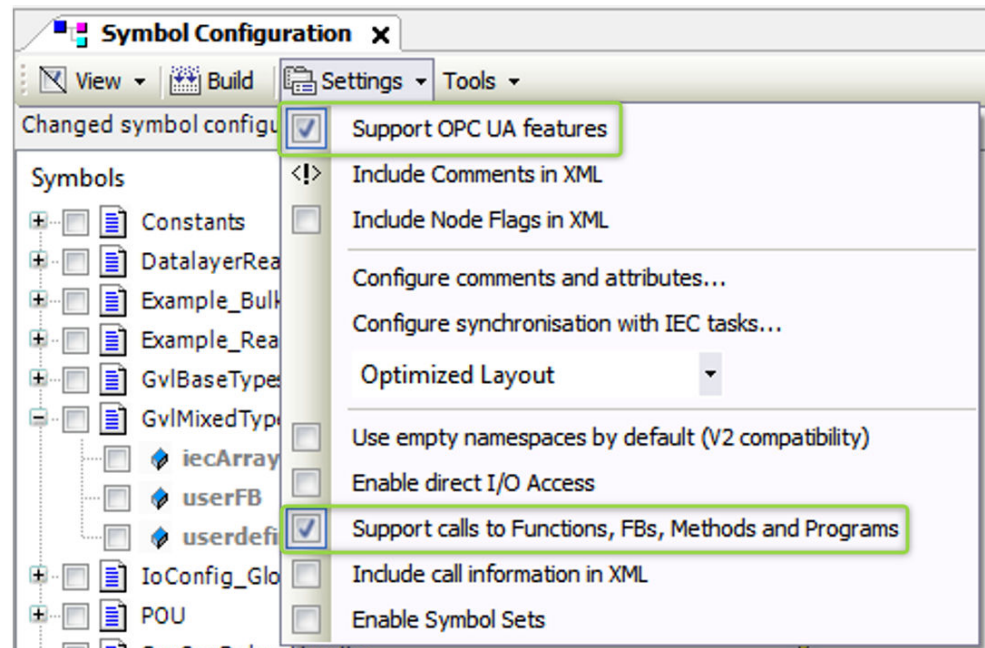


Abb. 11: Einstellungen der Symbolkonfiguration in ctrlX PLC Engineering

10.1 Verwendung von Strukturen

Um einen einfacheren Zugriff auf komplexe Datentypen zu ermöglichen, muss dem Data Layer auch der Datentyp bekannt gegeben werden. Wird im IEC-Programm eine neue Struktur angelegt, kann das Programm mit dieser Struktur einfach heruntergeladen werden.

Beispiel: Nutzerdefinierte Struktur mit 3 Elementen

Deklaration der nutzerdefinierten Struktur in PLC Engineering:

```

1  TYPE UserdefinedStruct :
2  STRUCT
3      structtest : BOOL;
4      structtest2 : BYTE;
5      structtest3 : INT;
6  END_STRUCT
7  END_TYPE

```

Nach dem Programm-Download auf das ctrlX-Gerät wird im Datalayer unter dem Knoten „types“ ein neuer Datentyp mit einem Flatbuffer-Schema in der ctrlX OS Web-Oberfläche angezeigt:

types

- apps
- common
- datalayer
 - diagnosis
 - health
 - plc
 - app
 - ApplicationArray
 - UserdefinedStruct**
 - app-date-and-time
 - app-idm-user-infos
 - app-idm-user-login
 - app-info
 - app-load-applications
 - app-memory-area
 - app-memory-area-access

UserdefinedStruct

Type information

```

1  {
2      "$schema": "https://json-schema.org/draft/2019-09/schema",
3      "definitions": {
4          "UserdefinedStruct": {
5              "type": "object",
6              "properties": {
7                  "structtest": {
8                      "type": "boolean",
9                      "default": false
10                 },
11                 "structtest2": {
12                     "type": "integer",
13                     "minimum": 0,
14                     "maximum": 255,
15                     "default": 0
16                 },
17             }
18         }
19     }

```

Metadata

Name	Value
Node Class	Type
Operations	read browse
Display format	Auto
References	readType: types/datalayer/reflection

JSON Flatbuffer-Schema:

```

{
  "$schema": "https://json-schema.org/draft/2019-09/schema",
  "definitions": {
    "UserdefinedStruct": {
      "type": "object",
      "properties": {
        "structtest": {
          "type": "boolean",
          "default": false
        },
        "structtest2": {
          "type": "integer",
          "minimum": 0,
          "maximum": 255,
          "default": 0
        }
      }
    }
  }
}

```

```

        "structtest3": {
            "type": "integer",
            "minimum": -32768,
            "maximum": 32767,
            "default": 0
        },
        "additionalProperties": false
    },
    "$ref": "#/definitions/UserdefinedStruct"
}

```

Zur üblichen Darstellung der Strukturvariable mit Hilfe von Unterknoten zu den einzelnen Elementen wird in den Metadaten unter „References“ der neue registrierte Datentyp im Datalayer verlinkt

userdefinedStruct

Value (object)

```

1 {
2   "structtest": true,
3   "structtest2": 15,
4   "structtest3": -1913
5 }
6

```

Write Discard

Child nodes

Metadata

Name	Value
Node Class	Variable
Operations	read write browse
Display format	Auto
References	readType: types/plc/app/ApplicationArray/UserdefinedStruct writeType: types/plc/app/ApplicationArray/UserdefinedStruct

Dadurch wird im „Value“-Bereich die Struktur aus dem IEC-Programm vollständig dargestellt.

Abhängig von den Zugriffsrechten read/write können diese Werte nun editiert und per Schreibbefehl im IEC-Programm geändert werden.

10.2 Verwendung von Arrays von Strukturen

Wird im Programm ein Array von einer Struktur erzeugt, ist die Darstellung recht ähnlich.

Beispiel: Array von einer nutzerdefinierten Struktur

```
UserArray: ARRAY[0..15] OF UserdefinedStruct :=
```

Für die Darstellung im Data Layer der ctrlX OS Web-Oberfläche wird automatisch ein Typ generiert, der den Wortlaut „ArrayOf“ vor der nutzerdefinierten Struktur stellt:

Type information

Value (object)

```

1 {
2   "$schema": "https://json-schema.org/draft/2019-09/schema",
3   "definitions": {
4     "ArrayOfUserdefinedStruct": {
5       "type": "object",
6       "properties": {
7         "value": {
8           "type": "array",
9           "items": {
10            "$ref": "#/definitions/UserdefinedStruct"
11          }
12        }
13      },
14      "additionalProperties": false
15    },
16    "UserdefinedStruct": {

```

Metadata

Name	Value
Node Class	Type
Operations	read browse
Display format	Auto
References	readType: types/datalayer/reflection

JSON Flatbuffer-Schema:

```

{
  "$schema": "https://json-schema.org/draft/2019-09/schema",
  "definitions": {
    "ArrayOfUserdefinedStruct": {
      "type": "object",
      "properties": {
        "value": {
          "type": "array",
          "items": {
            "$ref": "#/definitions/
UserdefinedStruct"
          }
        }
      },
      "additionalProperties": false
    },
    "UserdefinedStruct": {
      "type": "object",
      "properties": {
        "structtest": {
          "type": "boolean",
          "default": false
        },
        "structtest2": {

```

```

        "type": "integer",
        "minimum": 0,
        "maximum": 255,
        "default": 0
    },
    "structtest3": {
        "type": "integer",
        "minimum": -32768,
        "maximum": 32767,
        "default": 0
    }
},
"additionalProperties": false
},
"$ref": "#/definitions/ArrayOfUserdefinedStruct"
}

```

Ebenso wie für nutzerdefinierte Strukturen, gilt gleiches für die Verwendung von Arrays:

Beispiel: ctrlX OS Web-Oberfläche / Data Layer

The screenshot shows the ctrlX OS Web-Oberfläche / Data Layer interface. On the left, a tree view shows the project structure, with 'UserArray' selected under 'GvlMixedTypes'. The main area displays the 'UserArray' structure, which is an array of objects. The 'Value' field shows a JSON object with three properties: 'structtest', 'structtest2', and 'structtest3'. The 'Child nodes' section is empty. The 'Metadata' section shows the following information:

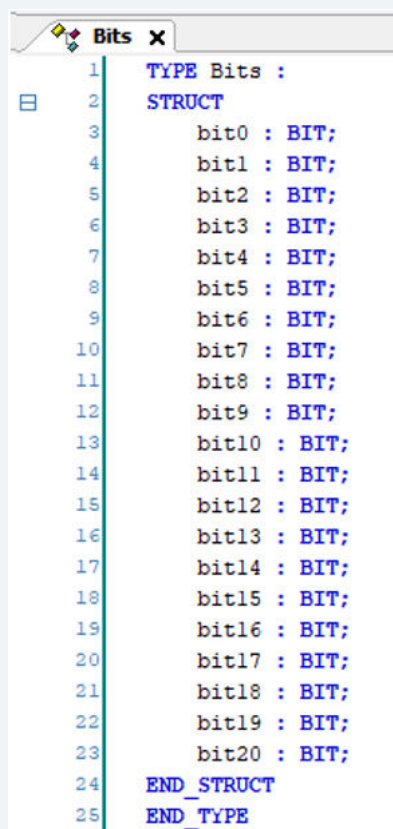
Name	Value
Node Class	Variable
Operations	read write browse
Display format	Auto
References	readType: types/plc/app/ApplicationArray/ArrayOfUserdefinedStruct writeType: types/plc/app/ApplicationArray/ArrayOfUserdefinedStruct

Das neu erstellte Flatbufferschema ist als Referenz hinterlegt und im Data Layer kann die Variable vollständig dargestellt werden.

10.3 Verwendung von Strukturen mit Bits

Ab Version PLC-V-0206 wird die Verwendung des Datentyp Bit innerhalb einer Struktur unterstützt.

Beispiel: Deklaration in ctrlX PLC Engineering



```
1  TYPE Bits :  
2  STRUCT  
3      bit0 : BIT;  
4      bit1 : BIT;  
5      bit2 : BIT;  
6      bit3 : BIT;  
7      bit4 : BIT;  
8      bit5 : BIT;  
9      bit6 : BIT;  
10     bit7 : BIT;  
11     bit8 : BIT;  
12     bit9 : BIT;  
13     bit10 : BIT;  
14     bit11 : BIT;  
15     bit12 : BIT;  
16     bit13 : BIT;  
17     bit14 : BIT;  
18     bit15 : BIT;  
19     bit16 : BIT;  
20     bit17 : BIT;  
21     bit18 : BIT;  
22     bit19 : BIT;  
23     bit20 : BIT;  
24  END_STRUCT  
25  END_TYPE
```

Diese Bits werden im Data Layer unter dem Knoten „types“ mit einem Flatbufferschema als Bool-Variablen registriert:

Beispiel: ctrlX OS Web-Oberfläche / Data Layer

- ^ types
 - > apps
 - > common
 - > datalayer 65
 - > diagnosis
 - > health
 - ^ plc
 - ^ app
 - ApplicationArray
 - Bits**
 - app-date-and-time
 - app-idm-user-infos
 - app-idm-user-login
 - app-info
 - app-load-applications
 - app-memory-area
 - app-memory-area-access
 - app-operation-state
 - app-project-info
 - app-reset
 - app-state
 - app-task-info
 - app-tasks-info

Bits

Type information

```

1  {
2    "$schema": "https://json-schema.org/draft/2019-09/schema",
3    "definitions": {
4      "Bits": {
5        "type": "object",
6        "properties": {
7          "bit0": {
8            "type": "boolean",
9            "default": false
10         },
11         "bit1": {
12           "type": "boolean",
13           "default": false
14         },
15         "bit2": {
16           "type": "boolean",

```

Metadata

Name	Value
Node Class	Type
Operations	read browse
Display format	Auto
References	readType: types/datalayer/reflection

Einzelne Bits werden auch als Knoten des Typs Bool dargestellt.

JSON Flatbuffer-Schema:

```

{
  "$schema": "https://json-schema.org/draft/2019-09/schema",
  "definitions": {
    "Bits": {
      "type": "object",
      "properties": {
        "bit0": {
          "type": "boolean",
          "default": false
        },
        "bit1": {
          "type": "boolean",
          "default": false
        },
        "bit2": {
          "type": "boolean",
          "default": false
        },
        "bit3": {
          "type": "boolean",
          "default": false
        },
        "bit4": {
          "type": "boolean",
          "default": false
        }
      }
    }
  }
}

```

```
"bit5": {  
    "type": "boolean",  
    "default": false  
},  
"bit6": {  
    "type": "boolean",  
    "default": false  
},  
"bit7": {  
    "type": "boolean",  
    "default": false  
},  
"bit8": {  
    "type": "boolean",  
    "default": false  
},  
"bit9": {  
    "type": "boolean",  
    "default": false  
},  
"bit10": {  
    "type": "boolean",  
    "default": false  
},  
"bit11": {  
    "type": "boolean",  
    "default": false  
},  
"bit12": {  
    "type": "boolean",  
    "default": false  
},  
"bit13": {  
    "type": "boolean",  
    "default": false  
},  
"bit14": {  
    "type": "boolean",  
    "default": false  
},  
"bit15": {  
    "type": "boolean",  
    "default": false  
},  
"bit16": {  
    "type": "boolean",  
    "default": false  
},  
"bit17": {  
    "type": "boolean",  
    "default": false  
},  
"bit18": {  
    "type": "boolean",  
    "default": false  
},  
"bit19": {
```



```

        "type": "boolean",
        "default": false
    },
    "bit20": {
        "type": "boolean",
        "default": false
    }
},
"additionalProperties": false
}
},
"$ref": "#/definitions/Bits"
}

```

Mit der Referenz des Flatbufferschema kann nun die gesamte Struktur der Bits angepasst werden, ohne jedes Bit einzeln abzufragen.

Beispiel: ctrlX OS Web-Oberfläche / Data Layer

The screenshot shows the ctrlX OS Web-Oberfläche / Data Layer interface. On the left is a file tree with the following structure:

- > datalayer
- > devices
- > diagnosis
- > framework
- ^ plc
 - > admin
 - ^ app
 - ^ ApplicationUserdefinedSymbols
 - > admin
 - ^ app
 - ^ mem
 - ^ sym
 - ^ GvlMixedTypes
 - > bitsTest (selected)
 - ^ mem
 - ^ sym
 - > scheduler
 - > script
 - > system
 - > trace
 - > types

On the right, the 'bitsTest' object is displayed with the following value (object):

```

1 {
2   "bit0": false,
3   "bit1": true,
4   "bit2": false,
5   "bit3": true,
6   "bit4": false,
7   "bit5": true,
8   "bit6": false,
9   "bit7": true,
10  "bit8": false,
11  "bit9": true,
12  "bit10": false,
13  "bit11": true,
14  "bit12": false,
15  "bit13": true,
16  "bit14": false,

```

Below the value, there are 'Write' and 'Discard' buttons. Under 'Child nodes', there are no nodes listed. Under 'Metadata', there is a table:

Name	Value
Node Class	Variable
Operations	read write browse
Display format	Auto
References	readType: types/plc/app/ApplicationUserdefinedSymbols/Bits writeType: types/plc/app/ApplicationUserdefinedSymbols/Bits

10.4 Verwendung von Enums

Enums wurden bisher als einfache Integer-Variablen behandelt.

Ab Version PLC-V-0206 werden ebenso für Enums Flatbufferschema angelegt, damit diese mit ihren eigenen Bezeichnungen verwendet werden können, wie im folgenden Beispiel gezeigt.

Beispiel: Deklaration Enum Testvariable in ctrlX PLC Engineering

```

1  TYPE EnumTest :
2  (
3      enumTestMember1 := 0,
4      enumTestMember2 := 1,
5      enumTestMember3 := 2
6  );
7  END_TYPE
8

```

Unter dem Knoten „types“ ist dieses Enum vollständig in einem Flatbuffer-schema beschrieben:

Beispiel: ctrlX OS Web-Oberfläche / Data Layer

- ^ types
 - > apps
 - > common
 - > datalayer 65
 - > diagnosis
 - > health
 - ^ plc
 - ^ app
 - ApplicationUserdefinedSymbols
 - EnumTest**
 - app-date-and-time
 - app-idm-user-infos
 - app-idm-user-login
 - app-info
 - app-load-applications
 - app-memory-area
 - app-memory-area-access
 - app-operation-state
 - app-project-info
 - app-reset
 - app-state
 - app-task-info
 - app-tasks-info

EnumTest

Type information

```

1  {
2      "$schema": "https://json-schema.org/draft/2019-09/schema",
3      "definitions": {
4          "EnumTest": {
5              "type": "string",
6              "enum": [
7                  "enumTestMember1",
8                  "enumTestMember2",
9                  "enumTestMember3"
10             ]
11          },
12          "": {
13              "type": "object",
14              "properties": {
15                  "EnumTest": {
16                      "$ref": "#/definitions/EnumTest",

```

Metadata

Name	Value
Node Class	Type
Operations	read browse
Display format	Auto
References	readType: types/datalayer/reflection

JSON Flatbuffer-Schema:

```
{
  "$schema": "https://json-schema.org/draft/2019-09/schema",
  "definitions": {
    "EnumTest": {
      "type": "string",
      "enum": [
        "enumTestMember1",
        "enumTestMember2",
        "enumTestMember3"
      ]
    },
    "": {
      "type": "object",
      "properties": {
        "EnumTest": {
          "$ref": "#/definitions/EnumTest",
          "default": "enumTestMember1"
        }
      },
      "additionalProperties": false
    }
  },
  "$ref": "#/definitions/"
}
```

Die Darstellung des Enums erfolgt nun im Knoten des Data Layer wie im folgenden Beispiel gezeigt:

Beispiel: ctrlX OS Web-Oberfläche / Data Layer

The screenshot shows the 'testEnum' node in the Data Layer. The value is set to 'enumTestMember2'. The metadata section is expanded, showing the following details:

Name	Value
Node Class	Variable
Operations	read write browse
Display format	Auto
References	readType: types/plc/app/ApplicationUserdefinedSymbols/EnumTest writeType: types/plc/app/ApplicationUserdefinedSymbols/EnumTest

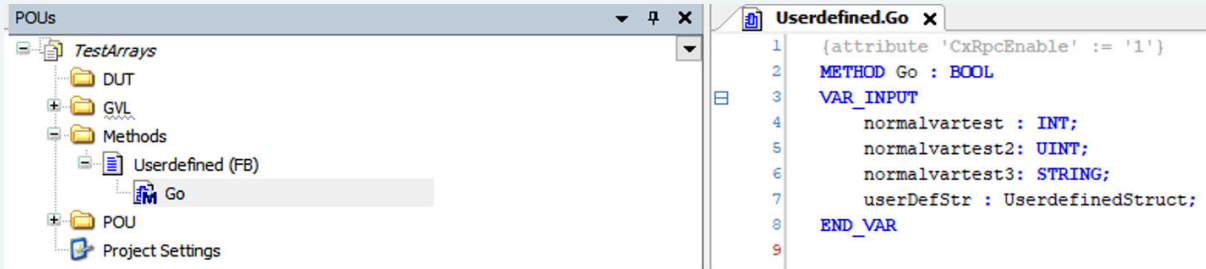
Es wird kein Integer-Wert mehr dargestellt, stattdessen ist es nun möglich direkt die Bezeichnung zu verwenden.

Ebenso kann auch ein Integer-Wert zugewiesen werden, welcher nach der „Write“-Anweisung jedoch wieder als Enum dargestellt wird.

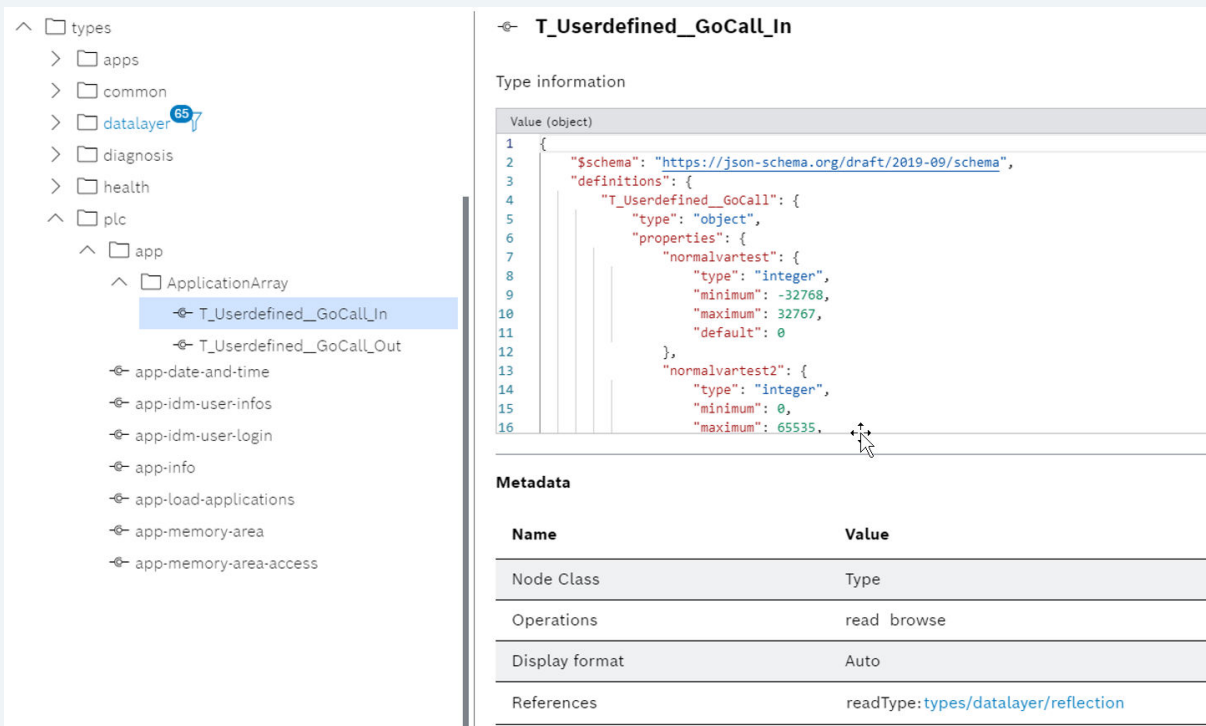
10.5 Methodenaufrufe

Wenn eine Methode mit Parametern erstellt wird, wird auch für diese Methode ein neuer Datentyp im Datalayer erstellt.

Beispiel: Deklaration in PLC Engineering



Da es sowohl Input- als auch Output-Variablen gibt, wird für die Variablenarten jeweils ein eigener Datentyp in Data Layer der ctrlX OS Web-Oberfläche angelegt:



JSON Flatbuffer-Schema:

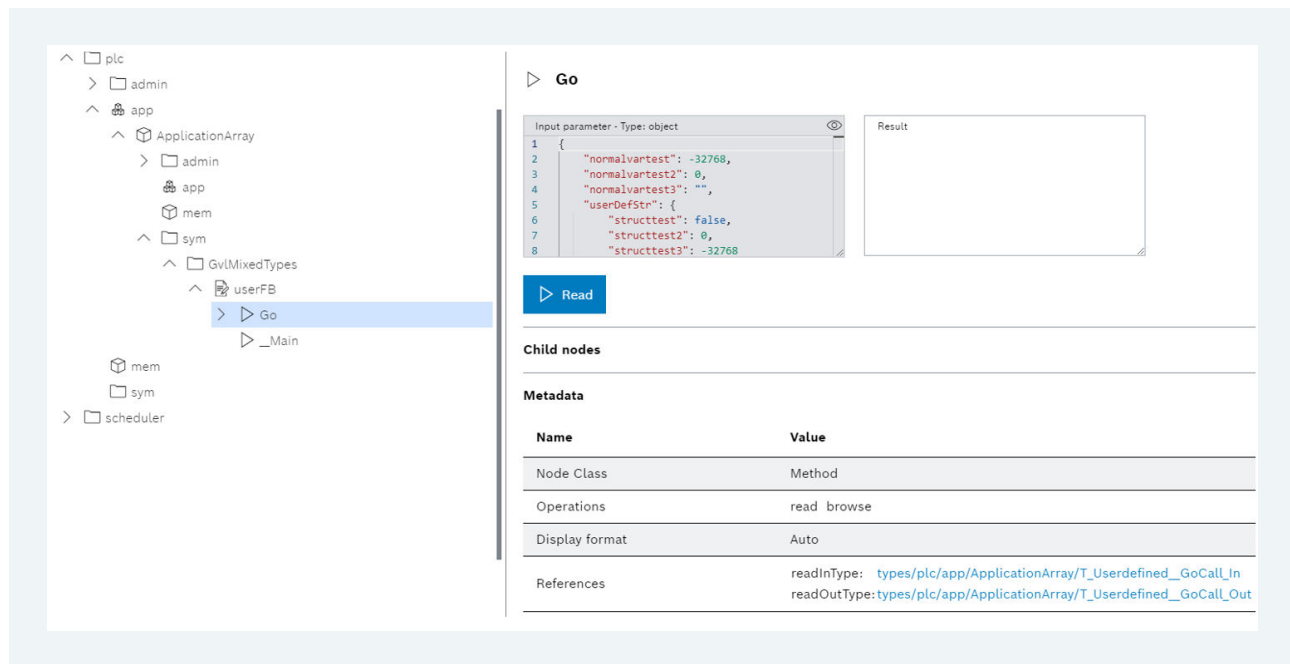
```

{
  "$schema": "https://json-schema.org/draft/2019-09/schema",
  "definitions": {
    "T_Userdefined_GoCall": {
      "type": "object",
      "properties": {
        "normalvartest": {
          "type": "integer",
          "minimum": -32768,
          "maximum": 32767,
          "default": 0
        },
        "normalvartest2": {
          "type": "integer",
          "minimum": 0,
          "maximum": 65535
        }
      }
    }
  }
}

```

```
        "normalvartest2": {
            "type": "integer",
            "minimum": 0,
            "maximum": 65535,
            "default": 0
        },
        "normalvartest3": {
            "type": "string"
        },
        "userDefStr": {
            "$ref": "#/definitions/UserdefinedStruct"
        }
    },
    "required": [
        "normalvartest3",
        "userDefStr"
    ],
    "additionalProperties": false
},
"UserdefinedStruct": {
    "type": "object",
    "properties": {
        "structtest": {
            "type": "boolean",
            "default": false
        },
        "structtest2": {
            "type": "integer",
            "minimum": 0,
            "maximum": 255,
            "default": 0
        },
        "structtest3": {
            "type": "integer",
            "minimum": -32768,
            "maximum": 32767,
            "default": 0
        }
    },
    "additionalProperties": false
}
},
"$ref": "#/definitions/T_Userdefined__GoCall"
}
```

Dadurch kann die Methode direkt mit allen Parametern aufgerufen werden, wie in folgendem Beispiel im Data Layer der ctrlX OS Web-Oberfläche:



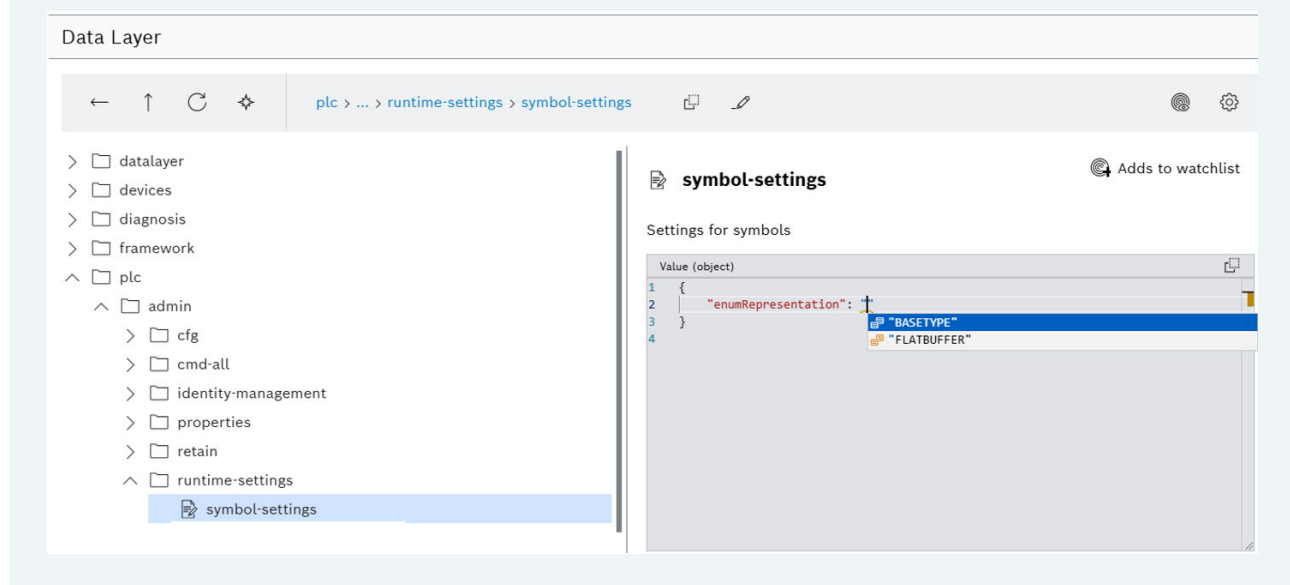
10.6 Einstellungsmöglichkeiten

10.6.1 Enum-Darstellung

Es ist möglich die Darstellung von ENUM-Datentypen zu ändern.

Variablen des Typs ENUM wurden bisher immer mit ihrem Basistypen wie im IEC-Programm angegeben dargestellt. Im Rahmen der Entwicklung wurde diese Darstellung insbesondere im Hinblick auf OPC UA geändert. Standardmäßig wird nun jeder ENUM-Datentyp als Flatbuffer hinterlegt und dargestellt. Dadurch kann mit den Elementnamen gearbeitet werden. Um weiterhin abwärtskompatibel zu bleiben, besteht die Möglichkeit die alte Darstellungsvariante weiterzuverwenden. Unter dem Pfad „plc/admin/runtime-settings/symbol-settings“ findet sich nun folgender Eintrag:

Data Layer: symbol-settings im Data Layer der ctrlX OS Web-Oberfläche



Mögliche Werte für den Eintrag „enumRepresentation“:

- **BASETYPE**: Es werden die Basisdatentypen aus dem ENUM zur Darstellung verwendet.
- **FLATBUFFER** (Standard): Es werden Flatbufferschemas zur Darstellung der ENUM-Datentypen verwendet.

Beispielsweise wird ein IEC-Typ DINT mit der Einstellung „BASETYPE“ wie folgt dargestellt:

Beispiel: ctrlX OS Web-Oberfläche / Data Layer

The screenshot shows the Data Layer interface. On the left, a tree view lists various IEC data types, with `iecEnumColorDint` selected. The main panel displays the details for `iecEnumColorDint`. The value is shown as `Value (int32)*` with the value `2`. Below the value field are `Write` and `Discard` buttons. The `Metadata` section shows the following information:

Name	Value
Node Class	Variable
Operations	read write browse
Display format	Auto

Mit der Option „FLATBUFFER“ findet man die Variable im Data Layer wie folgt vor:

Beispiel: ctrlX OS Web-Oberfläche / Data Layer

The screenshot shows the Data Layer interface. On the left, a tree view lists various IEC data types, with `iecEnumColorDint` selected. The main panel displays the details for `iecEnumColorDint`. The value is shown as `Value (object)` with the following JSON structure:

```

1 {
2   "EnumColorDINT": "blue"
3 }
4

```

Below the value field are `Write` and `Discard` buttons. The `Metadata` section shows the following information:

Name	Value
Node Class	Variable
Operations	read write browse
Display format	Auto

Optimierter Zugriff auf komplexe IEC-Datentypen über den Data Layer

10.6.2 Browsen von Array-Variablen über OPC UA

Für das Browsen von Array-Variablen über OPC UA wird ab der Version 2.6 in den Metadaten das Element "Displayname" verwendet. In Vorgängerversionen wurde als Displayname immer der Browsing-Name ausgewertet. Dies kann z.B. bei HMI Applikationen, die auf dieser Basis erstellt wurden, und über OPC UA Array-Variablen in dem SPS Projekt referenzieren, zu fehlerhaften Anzeigen führen.

Um eine solche, bestehende Anwendung nicht ändern zu müssen, ist es möglich, über den Pfad „plc/admin/runtime-settings/symbol-settings“ mittels des Parameters „ArrayDisplayname“ die Anzeige entsprechend zu konfigurieren.

Mögliche Werte für den Eintrag „ArrayDisplayname“:

- **ON** (Standard): Displaynamen über Metadaten veröffentlichen(Standard).
- **OFF** : Displaynamen nicht angeben (Abbildung der Arrayvariablen kompatibel zu Versionen < PLC-V-0206).

10.7 Einschränkungen

Wenn Strukturen oder Arrays deklariert werden, für die kein Flatbuffer-Schema erzeugt werden kann, wird unter den Datentypen kein neuer Typ hinzugefügt. In diesem Fall werden im Logbuch des ctrlX-Geräts entsprechende Fehlermeldungen angezeigt.

Folgende Einschränkungen gelten zum Zeitpunkt der Dokumentationsausgabe:

- Strukturen mit Bits dürfen nicht mit anderen Typen gemischt werden. Diese Strukturen funktionieren nur als reine Bit-Strukturen.
- Methodenaufrufe mit Strukturen die Bits enthalten werden nicht unterstützt.
- Es werden keine Pointer in Strukturen oder Arrays unterstützt (Referenzen sind möglich).

11 Taskkonfiguration

11.1 Allgemeine Informationen zur Taskkonfiguration

Die Verwaltung der SPS-Anwendertasks erfolgt über den Scheduler des SPS-Laufzeitsystems. Der Scheduler des SPS-Laufzeitsystems ist nicht mit dem ctrlX OS-Scheduler zu verwechseln und wird fest mit einem Intervall von einer Millisekunde getaktet. In diesem Abstand prüft der Scheduler zyklisch, welche SPS -Tasks vom Typ "Zyklisch", "Ereignis", "Freilaufend" und "Status" zur Abarbeitung freigegeben werden. Die zyklischen Tasks können in der Taskkonfiguration daher nur mit einer minimalen Zykluszeit und als Vielfaches von einer Millisekunde konfiguriert werden.

Diese Einschränkung gilt auch für die Konfiguration und Behandlung der Zeiten für den Taskwatchdog. Da der SPS-Scheduler im genannten Takt auch zyklisch die Laufzeiten aller aktuell laufenden SPS-Tasks, auch der Ereignisgesteuerten-Tasks, auf die Überschreitung der konfigurierten Watchdog-Zeiten prüft, können hierfür ebenfalls nur Werte als Vielfaches von einer Millisekunde vergeben werden.

Werden Zykluszeiten kleiner einer Millisekunde benötigt, kann dies über den Tasktyp "Extern" umgesetzt werden. Der Scheduler des ctrlX OS Systems erlaubt auch die Konfiguration von Tasks mit Zykluszeiten kleiner einer Millisekunde, in deren Kontext ein Event generiert wird, das wiederum den Start der zugewiesenen SPS-Task signalisiert.

Hinweis: Der Startzeitpunkt und das Laufzeitverhalten einer SPS Task ist abhängig von der konfigurierten Priorität. Auch wenn der SPS Scheduler die Abarbeitung einer Task freigegeben hat, kann diese nur dann laufen, so lange keine andere Task im System mit einer "höheren" Priorität auf dem zugewiesenen Prozessorkern aktiv ist.

11.2 Ereignisquellen für extern-ereignisgesteuerte SPS-Tasks

Der ctrlX Scheduler bietet zwei Arten der Programm Konfiguration:

- Automatische Programm-Konfiguration (neu ab Version PLC-V-0206).
- Frei konfigurierbare, externe Ereignisquellen.

Weiterführende Informationen zu Scheduling & Timing siehe: ➔ [ctrlX OS Dokumentation](#)

11.3 Unterstützung der Automatischen Programm-Konfiguration des ctrlX Schedulers

Ab Version PLC-V-0206 unterstützt die PLC-App die automatische Programm-Konfiguration des ctrlX Schedulers.

Externe Ereignisse in ctrlX PLC Engineering zur Task-Steuerung verwenden

ctrlX PLC Engineering stellt im Konfigurationsdialog einer SPS Task für den Task-Typ "**Extern**" die folgenden neuen externen Ereignisse zur Verfügung:

Externes Ereignis	Beschreibung
AFTER_CTRLX_ETHERCAT_MASTER	Dieses Ereignis triggert einen SPS-Task, nachdem der ctrlX Ethercat-Master aktuelle Ausgangswerte bereitgestellt hat.
AFTER_CTRLX_MOTION_VALUE_GENERATION	Dieses Ereignis triggert einen SPS-Task, nachdem die ctrlX Motion aktuelle Ausgangswerte bereitgestellt hat.
AFTER_CTRLX_PROFINET_DEVICE	Dieses Ereignis triggert einen SPS-Task, nachdem das ctrlX Profinet Device aktuelle Ausgangswerte bereitgestellt hat.

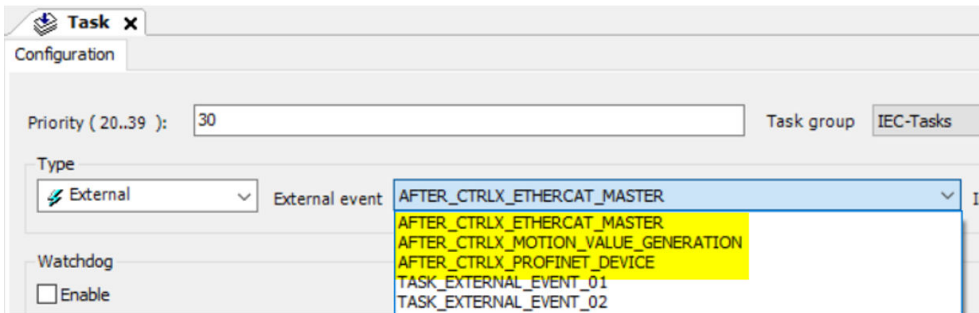


Abb. 12: Externe Ereignisse in ctrlX PLC Engineering

Wenn der ctrlX Scheduler im Modus „Automatische Programm Konfiguration“ ist und ein SPS-Programm heruntergeladen wird, das Tasks enthält, die eines der neuen externen Ereignisse verwenden, dann wird im Scheduler automatisch ein entsprechendes „Callable“ angelegt, das den SPS-Task triggert. D.h. der Anwender muss im Scheduler selbst nichts mehr konfigurieren.

Beispiel: Ein SPS-Task direkt nach der Motion-Wert-Berechnung triggern

1. ➤ Legen Sie im SPS-Programm einen extern ereignisgesteuerten Task an, der mit dem externen Ereignis „AFTER_CTRLX_MOTION_VALUE_GENERATION“ verknüpft ist.

Info: Die Motion-Wert-Berechnung wird als „callable“ im ctrlX Scheduler Task „ctrlXAutomation“ ausgeführt.

2. ➤ Laden Sie das SPS-Programm in die Steuerung.

- ➔ Beim Herunterladen des SPS-Programms wird automatisch im Scheduler ein „callable“ für die PLC angelegt, das nach dem Motion-„callable“ läuft und den PLC-Task triggert.

Folgende „callables“ werden in einer Scheduler Task angelegt, wenn ein entsprechendes externes Event im SPS-Programm verwendet wird:

Name des „callables“	Scheduler task
plc-after-ctrlx-ethercat-master	ctrlXAutomation
plc-after-ctrlx-motion-value-generation	ctrlXAutomation
plc-after-ctrlx-profinet-device	profinetdevice

Die „callables“ verwenden die folgenden Synchronisationspunkte, um die gewünschte Reihenfolge aller „callables“ in einem Scheduler Task herzustellen:

Name des „callables“	„Bevor“ Synchronisationspunkte	„Danach“ Synchronisationspunkte
plc-after-ctrlx-ethercat-master	axisprofile-adaption-entry motion-entry oscilloscope-entry	field buses-ethercat-master-exit
plc-after-ctrlx-motion-value-generation	axisprofile-distribution-entry oscilloscope-entry	motion-value-generation-exit
plc-after-ctrlx-profinet-device		field buses-profinet-device-exit

11.4 Frei konfigurierbare, externe Ereignisquellen

ctrlX PLC Engineering stellt im Konfigurationsdialog einer SPS-Task mehrere Typen zur Auswahl. Der Task-Typ "Extern" stellt die externen Ereignisse TASK_EXTERNAL_EVENT_01 bis TASK_EXTERNAL_EVENT_10 zur Verfügung. Diese Ereignisse sind auf der Steuerung zunächst nicht zugeordnet. Die folgende Handlungsanweisung beschreibt, wie externe Events konfiguriert werden können.



Bis ctrlX PLC App-Version PLC-V-0114 wurde TASK_EXTERNAL_EVENT_07 automatisch angelegt.

Ab ctrlX PLC App-Version PLC-V-0116 müssen Events wie nachfolgend beschrieben konfiguriert werden.

Konfiguration eines externen Ereignisses

1. ➤ Schalten Sie das ctrlX-Gerät über die Statusschaltflächen im Header in den Betriebszustand "Setup"
2. ➤ Rufen Sie über die Seitennavigation das Fenster „Scheduler“ auf.

3. ➤ Wählen Sie im Fenster eine Task aus, in deren Kontext das Ereignis generiert werden soll.
Wenn Sie Ihre PLC-Task mit der Motion- und der EtherCAT-Kommunikation synchronisieren wollen, wählen Sie bitte für das weitere Vorgehen die Task "ctrlXAutomation".
Alternativ können Sie auch eine neue Task mit den gewünschten Eigenschaften anlegen.
4. ➤ Öffnen Sie über die Schaltfläche  die Liste der Callables auf, die im zugehörigen Task-Kontext aufgerufen werden.
5. ➤ Öffnen Sie über die Schaltfläche  den Dialog zum Hinzufügen eines Callables und wählen Sie **plc-external-event** mit dem gewünschten Run-Index.
➔ Der Dialog zur Auswahl des Ereignisses wird angezeigt und bietet die Argumente **TASK_EXTERNAL_EVENT_01** bis **TASK_EXTERNAL_EVENT_10** zur Auswahl an.
6. ➤ Wählen Sie das gewünschte Ereignis aus und löschen Sie alle anderen Einträge, so dass nur noch ein Argument angezeigt wird.
Die Auswahl mehrerer Argumente bzw. die Mehrfachnutzung eines Arguments in verschiedenen Tasks ist nicht möglich.
7. ➤ Schalten Sie das ctrlX-Gerät wieder in den Betriebszustand "Operating".
➔ Die Ereignisse werden jetzt generiert.
8. ➤ Weisen Sie abschließend über die Task-Konfiguration in ctrlX PLC Engineering das zuvor konfigurierte Ereignis der gewünschten SPS Task zu.
9. ➤ Laden Sie Ihr Programm auf die Steuerung und prüfen Sie, ob die SPS-Task aufgerufen wird.

Anwendungsbeispiel

Ein typischer Anwendungsfall ist beispielsweise die Synchronisation einer SPS-Task mit dem EtherCAT-Master.

Die Synchronisation wird erreicht, indem die Task "ctrlXAutomation" um ein Callable des Typs "plc-external-event" mit einem höheren Run-Index erweitert wird. So wird zunächst der EtherCAT-Master bearbeitet und anschließend das externe Ereignis zur Bearbeitung der SPS-Task erzeugt.

12 Erstellen von externen POEs

Neben der Erstellung von POEs (Programm-Organisationseinheiten) in den Eingabefachsprachen der IEC 6-1131 können diese auch „extern“ implementiert werden. Der Unterschied zu programminternen POEs besteht darin, dass zwar die Definition der POE-Schnittstellen wie gewohnt in IEC, die Implementierung der POEs in C oder C++ jedoch im SPS Laufzeitsystems erfolgt.

Wenn externe POEs im Programm verwendet werden, erfolgt das zugehörige "Linken" erst beim Download auf das Zielsystem. Wenn externe POEs nicht aufgelöst werden können, meldet das SPS Laufzeitsystem hierfür den Fehler „Unresolved external ...“.

Weitere Hinweise zur Erstellung von extern implementierten POEs siehe:
➔ [Web-Dokumentation](#)



Zur Erstellung von C/C++ Funktionen für ctrlX OS Steuerungen nutzen Sie bitte das ctrlX OS SDK (Software Development Kit), siehe hierzu folgende Web-Links:

➔ [ctrlX SDK How-To](#)

➔ [ctrlX SDK auf GitHub](#)

13 ctrlX Bedienoberfläche – Elemente

13.1 Navigation

13.1.1 Seitennavigation – PLC App

Mit der Installation der PLC App wird die Web-Oberfläche des ctrlX-Geräts um folgende Einträge ergänzt:

Seitennavigation – Fenster PLC

Das Fenster „PLC“ dient zur Verwaltung der SPS-Anwendungen (Applikationen) auf dem ctrlX-Gerät, siehe:

➔ [Fenster – „PLC“](#)

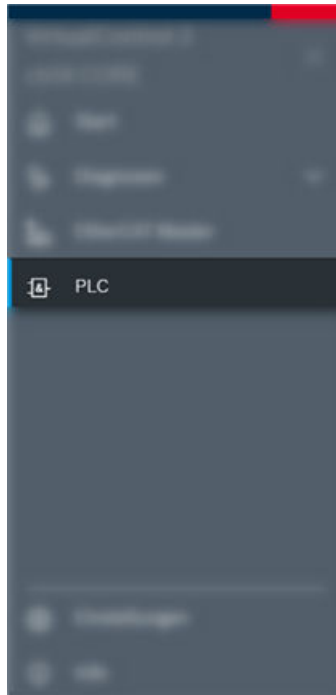


Abb. 13: Seitennavigation - „PLC“

Widget im Startfenster der Web-Oberfläche

Das Widget „SPS“ zeigt Informationen zum Betriebszustand der projektierten SPS-Anwendungen an und bietet Direktaufrufe zu folgenden Bereichen:

- Fenster „PLC“
- Software-Tool ctrlX PLC Engineering zur Erstellung des SPS-Projekts

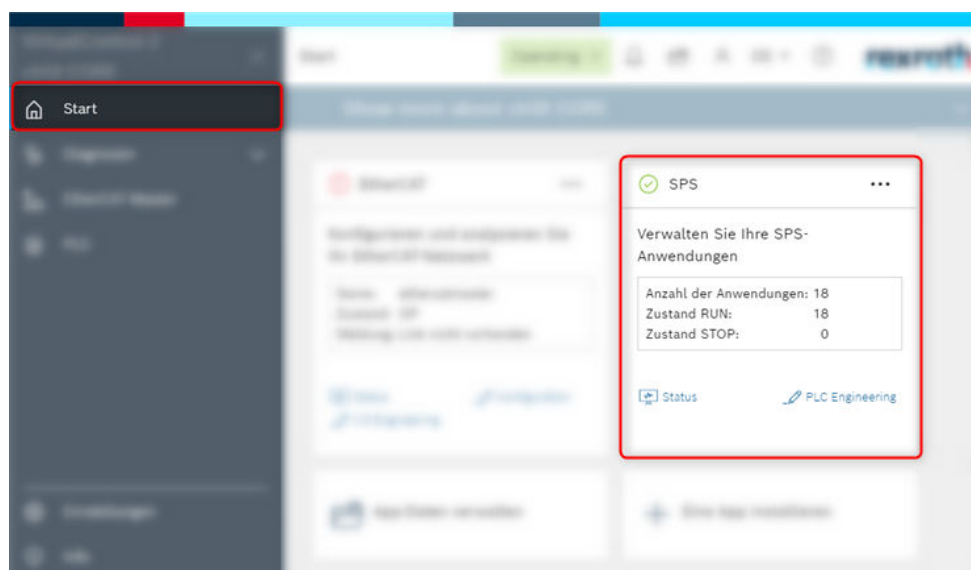


Abb. 14: Widget „SPS“

13.2 Fenster

13.2.1 Fenster – „PLC“

Das Fenster „PLC“ dient zur Verwaltung der SPS-Anwendungen (Applikationen) auf dem ctrlX-Gerät.

Über den Header im oberen Fensterbereich kann der Betriebszustand aller konfigurierten Anwendungen zentral gesteuert und bei Bedarf zurückgesetzt werden, siehe:

➔ [Header – „PLC“](#)

Weitere Steuerungsmöglichkeiten und Informationen werden unterhalb des Header in folgenden Registerkarten angeboten:

- **Registerkarte „Anwendungen“**
Zeigt alle Anwendungen, die auf dem ctrlX-Gerät konfiguriert bzw. geladen sind und bietet Schaltflächen zur Kommandierung einzelner Anwendungen (Start / Stop / Reset), siehe:
➔ [Registerkarte – „Anwendungen“](#)
- **Registerkarte „Boot-Anwendungen“**
Bietet eine tabellarische Auflistung aller Boot-Anwendungen, die auf dem ctrlX-Gerät konfiguriert sind, sowie deren Erstellungszeitpunkt, siehe:
➔ [Registerkarte – „Boot-Anwendungen“](#)
- **Registerkarte „Projektinformationen“**
Liefert generelle Informationen über das SPS-Projekt in Vergleichsdarstellung (Ziel-Applikation vs. Quell-Applikation), siehe:
➔ [Registerkarte – „Projektinformationen“](#)

Aufruf

In der ctrlX OS Web-Oberfläche:

„[Seitennavigation](#) → PLC“

Anwendung konfigurieren

Direkt nach der Installation der PLC App ist noch keine SPS-Anwendung auf dem ctrlX-Gerät konfiguriert.

Die Konfiguration einer SPS-Anwendung erfolgt im Software-Tool ctrlX PLC Engineering, das Teil der ctrlX WORKS-Installation auf einem Engineering-PC ist.

Um ctrlX PLC Engineering zu starten klicken Sie auf die Schaltfläche  im Header.

Beim erstmaligen Start von ctrlX PLC Engineering wird automatisch eine Projektvorlage im Gerätebaum angelegt, die kompatibel zur verwendeten ctrlX-Hardware ist. Die Projektvorlage besitzt bereits eine Objektstruktur, in der eine Anwendung (Applikation) vorkonfiguriert ist. Die Anwendung wird beim Onlinegehen auf das ctrlX-Gerät geladen und nach dem Projektdownload in der Registerkarte „Anwendungen“ angezeigt, siehe [↗ Registerkarte – „Anwendungen“](#).

Weitere Anwendungen können im Gerätebaum von ctrlX PLC Engineering hinzugefügt und auf das ctrlX-Gerät geladen werden.

Einschränkungen

Die Applikationsvariante "Kindapplikation" wird nicht unterstützt.

13.3 Header

13.3.1 Header – „PLC“

Der Header im Fenster „PLC“ dient zur zentralen Steuerung der SPS-Applikationen auf dem ctrlX-Gerät.

Funktionen

- Betriebszustand der Applikationen steuern
- Reset der Applikationen durchführen
- Konfiguration der Retain-Datenspeichergröße
- Aufruf von ctrlX PLC Engineering zur Erstellung von SPS-Projekten

Aufruf

In der ctrlX OS Web-Oberfläche:

„*Seitennavigation* → PLC“

Beschreibung

Oberflächenelement	Beschreibung
	Anwendungen starten
	Anwendungen stoppen
	Reset aller Anwendungen mit folgenden Reset-Optionen: • Reset warm • Reset kalt • Reset Ursprung
	Aufruf von ctrlX PLC Engineering zur Erstellung des SPS-Projekts
	Öffnet den Dialog „SPS-Retain-Speicher zuweisen (Bytes)“ zur Änderung der Speichergröße, siehe ↗ Größe des Speichers für remanente Daten ändern

Weiterführende Informationen

[↗ Fenster – „PLC“](#)

13.4 Registerkarten

13.4.1 Registerkarte – „Anwendungen“

Die Registerkarte bietet eine tabellarische Auflistung aller SPS-Anwendungen (Applikation), die auf dem ctrlX-Gerät konfiguriert bzw. geladen sind. Zu jeder gelisteten Anwendung wird der aktuelle Betriebszustand und gegebenenfalls anstehende Diagnosemeldungen angezeigt. Über Schaltflächen kann der Betriebszustand einzelner Anwendungen umgeschaltet oder ein Reset durchgeführt werden, siehe Optionen unten.

Aufruf

In der ctrlX OS Web-Oberfläche:

„Seitennavigation → PLC → Registerkarte "Anwendungen"“

Beschreibung

Tabellenelement	Beschreibung
„Name“	Name der SPS-Applikation
„Zustand“	Betriebszustand der SPS-Applikation: • „RUN“: SPS-Applikation ist gestartet • „STOP“: SPS-Applikation ist gestoppt
„Diagnosen“	Anzeige anstehender Diagnosemeldungen: • „Exception“ = Die Anwendung ist im Fehlerzustand. Maßnahme: Für weitere Details öffnen Sie bitte ctrlX PLC Engineering und prüfen Sie das "SPS-Logbuch und den "Aufrufbaum". • „ForceVariablesActive“ = Die Funktion "Werte forcen" ist in der Anwendung aktiv. Maßnahme: Um das Forcen zu beenden führen Sie bitte In ctrlX PLC Engineering den Befehl „Forcen für alle Werte aufheben“ aus. • „FlowControlActive“ = In der Anwendung ist der Modus "Ablaufkontrolle" aktiv. Maßnahme: Um die Ablaufkontrolle zu deaktivieren führen Sie bitte in ctrlX PLC Engineering den Befehl "Ablaufkontrolle" aus. • „RetainMismatch“ = Anwendung mit "Retain Mismatch" geladen. Maßnahme: Zum Ausführen der Anwendung ist ein "Reset kalt" erforderlich, siehe "Aktionen" unten.
„Aktionen“	Schaltflächen bzw. Operationen zur Steuerung des Betriebszustands: • 1. ▷ = Anwendung starten • 2. □ = Anwendung stoppen • 3. ⌂ = Anwendung zurücksetzen mit folgenden Optionen: – 3.1 Reset warm – 3.2 Reset kalt – 3.3 Reset Ursprung Wichtiger Hinweis: Die Operationen 2 und 3 sind ab Version PLC-V-0304 asynchrone Dienste mit einem standardmäßigen Timeout von 30000 ms. Der Timeout dient dazu, den Dienst abubrechen, wenn er die Aufgabe nicht rechtzeitig erledigen kann. Über den Data Layer-Knoten [plc/admin/runtime-settings/async-timeout-settings] kann der Wert für den Timeout bei Bedarf angepasst werden, beispielsweise bei hohen Zykluszeiten im SPS-Programm.

Verwandte Themen

➔ [Fenster – „PLC“](#)

13.4.2 Registerkarte – „Boot-Anwendungen“

Die Registerkarte bietet eine tabellarische Auflistung aller Boot-Anwendungen, die auf dem ctrlX-Gerät konfiguriert sind und deren Erstellungszeitpunkt.

Aufruf

In der ctrlX OS Web-Oberfläche:

„*Seitennavigation* → *PLC* → *Registerkarte "Boot-Anwendungen"*“

Beschreibung

Tabellenelement	Beschreibung
„Name“	Name der Boot-Anwendung
„Erstellt“	Erstellungszeitpunkt der Boot-Anwendung (Datum / Zeit)

Verwandte Themen

➔ [Fenster – „PLC“](#)

13.4.3 Registerkarte – „Projektinformationen“

Die Registerkarte liefert generelle Informationen über das SPS-Projekt in einer tabellarischen Vergleichsdarstellung zwischen dem aktuell geladenen Projekt und dem zuletzt geladenen Quellprojekt.

- Tabellenspalte „Zuletzt Geladen“
Liefert Informationen von der zuletzt auf die Steuerung geladenen Anwendung
- Tabellenspalte „Synchronisation“
Liefert Informationen von den zuletzt auf die Steuerung geladenen Projektquellen

Die Vergleichsansicht beinhaltet folgende Angaben:

- Projektname
- Datum der letzten Änderung
- Angaben zum Engineering-PC / Benutzername

Aufruf

In der ctrlX OS Web-Oberfläche:

„*Seitennavigation* → *PLC* → *Registerkarte "Projektinformationen"*“

Beschreibung

Tabellenelement	Tabellenspalte „Zuletzt Geladen“	Tabellenspalte „Synchronisation“
„Name“	Name des Projekts auf dem ctrlX-Gerät	Name des Synchronisierten-Projekts
„Letzte Änderung“	Datum und Uhrzeit der letzten Änderungen an den Anwendungen des Zielprojekts.	Datum und Uhrzeit der letzten Änderungen an den Anwendungen des Quellprojekts.

Tabellenelement	Tabellenspalte „Zuletzt Geladen“	Tabellenspalte „Synchronisation“
„Maschinenname“	Name des Engineering-PC, von dem aus eine Anwendung heruntergeladen wurde.	
„Benutzername“	Benutzername des Engineering-PC, der zum Laden der Anwendung verwendet wurde.	

Verwandte Themen
 ➔ Fenster – „PLC“

14 Weiterführende Dokumentationen

14.1 Übersicht

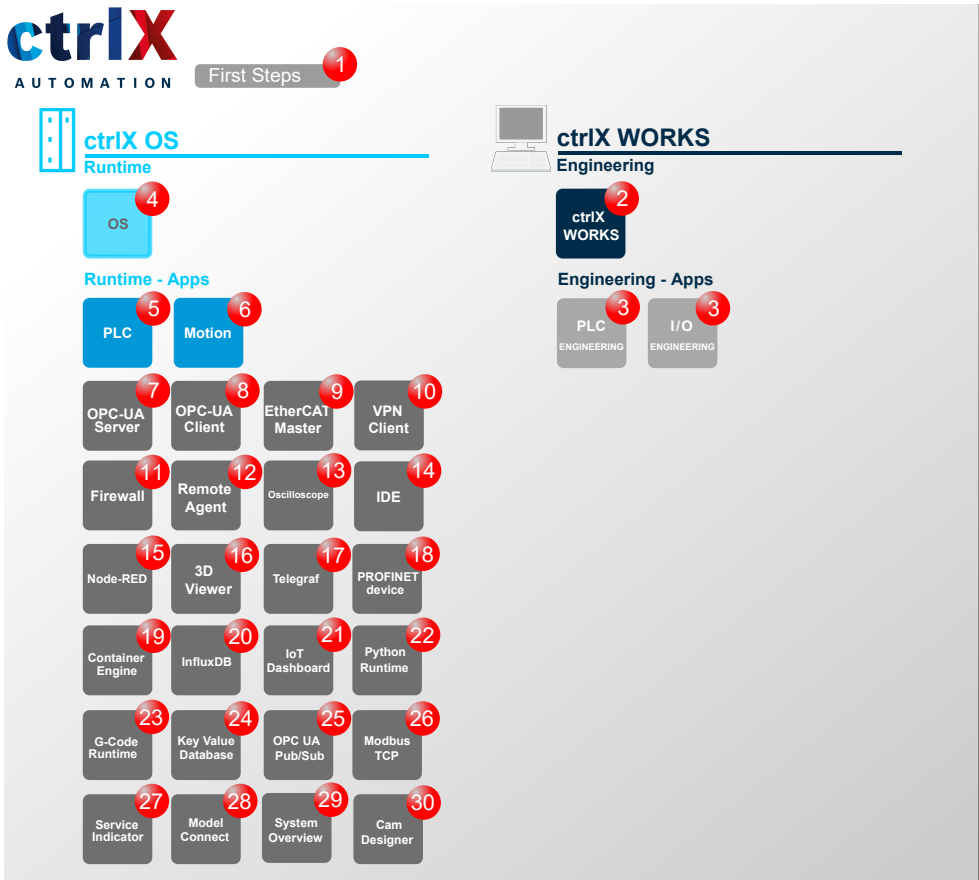


Abb. 15: Übersicht der weiterführenden Dokumentationen

14.2 ctrlX AUTOMATION

Nr.	Dokumentation
1	ctrlX WORKS - Erste Schritte Quick Start Guide ➔ Link zur Web-Dokumentation Bestellinformationen: • DOK-XWORKS-F*STEP*****-QU01-DE-P • R911403759

14.3 ctrlX WORKS

Nr.	Dokumentation
2	ctrlX WORKS - Basissystem 02VRS Anwendungsbeschreibung ↗ Link zur Web-Dokumentation Bestellinformationen: • DOK-XWORKS-WRK***V02**-APRS-DE-P • R911421575
3	ctrlX PLC Engineering - SPS-Programmiersystem 02VRS Anwendungsbeschreibung ↗ Link zur Web-Dokumentation Bestellinformationen: • DOK-XPLC**-ENG*****V02-APRS-DE-P • R911421577
3	ctrlX PLC Engineering - SPS-Bibliotheken 02VRS Referenz ↗ Link zur Web-Dokumentation Bestellinformationen: • DOK-XPLC**-LIB***V02**-RERS-DE-P • R911421579

14.4 ctrlX OS

Nr.	Dokumentation
4	ctrlX OS - Betriebssystem für ctrlX CORE-Steuerungsgeräte 02VRS Anwendungsbeschreibung ↗ Link zur Web-Dokumentation Bestellinformationen: • DOK-XCORE*-XCR***V02**-APRS-DE-P • R911421589
	ctrlX OS - Knoten des Data Layer 02VRS Referenz ↗ Link zur Web-Dokumentation Bestellinformationen: • DOK-XCORE*-DL****V02**-RERS-DE-P • R911421591
	ctrlX OS - Diagnosen 02VRS Referenz ↗ Link zur Web-Dokumentation Bestellinformationen: • DOK-XCORE*-DIAG**V02**-RERS-DE-P • R911421593

14.5 ctrlX OS Apps

Nr.	Dokumentation
5	PLC App - SPS-Laufzeitumgebung für ctrlX OS 02VRS Anwendungsbeschreibung ↪ Link zur Web-Dokumentation Bestellinformationen: • DOK-XCORE*-PLC***V02**-APRS-DE-P • R911421585
6	Motion App - Motion-Laufzeitumgebung für ctrlX CORE 02VRS Anwendungsbeschreibung ↪ Link zur Web-Dokumentation Bestellinformationen: • DOK-XCORE*-MOT***V02**-APRS-DE-P • R911421609
7	OPC UA Server App - OPC UA Server für ctrlX OS 02VRS Anwendungsbeschreibung ↪ Link zur Web-Dokumentation Bestellinformationen: • DOK-XCORE*-UAS***V02**-APRS-DE-P • R911421597
8	OPC UA Client App - OPC UA Client für ctrlX OS 02VRS Anwendungsbeschreibung ↪ Link zur Web-Dokumentation Bestellinformationen: • DOK-XCORE*-UAC***V02**-APRS-DE-P • R911421599
9	EtherCAT Master App - EtherCAT Master für ctrlX OS 02VRS Anwendungsbeschreibung ↪ Link zur Web-Dokumentation Bestellinformationen: • DOK-XCORE*-ECM***V02**-APRS-DE-P • R911421603
10	VPN Client App - Fernwartungssoftware für ctrlX OS 02VRS Anwendungsbeschreibung ↪ Link zur Web-Dokumentation Bestellinformationen: • DOK-XCORE*-VPN***V02**-APRS-DE-P • R911421595
11	Firewall App - Security Funktionen für ctrlX OS 02VRS Anwendungsbeschreibung ↪ Link zur Web-Dokumentation Bestellinformationen: • DOK-XCORE*-FRW***V02**-APRS-DE-P • R911421605

Nr.	Dokumentation
12	Remote Agent App - ctrlX Device Portal-Anbindung für ctrlX OS-Geräte 02VRS Anwendungsbeschreibung ↗ Link zur Web-Dokumentation Bestellinformationen: • DOK-XCORE*-RMA***V02**-APRS-DE-P • R911421607
13	Oscilloscope App - Oszilloskopfunktion für ctrlX OS-Geräte 02VRS Anwendungsbeschreibung ↗ Link zur Web-Dokumentation Bestellinformationen: • DOK-XCORE*-OSC***V02**-APRS-DE-P • R911421588
14	IDE App - Integrated Development Environment 02VRS Anwendungsbeschreibung ↗ Link zur Web-Dokumentation Bestellinformationen: • DOK-XCORE*-IDE***V02**-APRS-DE-P • R911421611
15	Node-RED App - Grafische Programmierung für ctrlX OS 02VRS Anwendungsbeschreibung ↗ Link zur Web-Dokumentation Bestellinformationen: • DOK-XCORE*-NODERED*V02-APRS-DE-P • R911421583
16	3D Viewer App - Browserbasierte 3D-Kinematik-Simulation für ctrlX OS 02VRS Anwendungsbeschreibung ↗ Link zur Web-Dokumentation Bestellinformationen: • DOK-XCORE*-3DV***V02**-APRS-DE-P • R911421614
17	Telegraf App - Server-Agent zum Sammeln von Daten im Data Layer 02VRS Anwendungsbeschreibung ↗ Link zur Web-Dokumentation Bestellinformationen: • DOK-XCORE*-TSA***V02**-APRS-DE-P • R911421622
18	PROFINET Device App - PROFINET Device für ctrlX OS 02VRS Anwendungsbeschreibung ↗ Link zur Web-Dokumentation Bestellinformationen: • DOK-XCORE*-PROFINETV02-APRS-DE-P • R911421616

Nr.	Dokumentation
19	Container Engine App - Verwendung von Docker® Images auf ctrlX OS 02VRS Anwendungsbeschreibung ↗ Link zur Web-Dokumentation Bestellinformationen: • DOK-XCORE*-DOE***V02**-APRS-DE-P • R911421618
20	InfluxDB App - Influx-Datenbankanbindung für ctrlX OS 02VRS Anwendungsbeschreibung ↗ Link zur Web-Dokumentation Bestellinformationen: • DOK-XCORE*-IDB***V02**-APRS-DE-P • R911421624
21	IoT Dashboard App - Datenvisualisierung in dynamischen, interaktiven Dashboards 02VRS Anwendungsbeschreibung ↗ Link zur Web-Dokumentation Bestellinformationen: • DOK-XCORE*-GDB***V02**-APRS-DE-P • R911421632
22	Python Runtime App - Python-Laufzeitumgebung für ctrlX CORE 02VRS Anwendungsbeschreibung ↗ Link zur Web-Dokumentation Bestellinformationen: • DOK-XCORE*-PYR***V02**-APRS-DE-P • R911421628
23	G-Code Runtime App - G-Code Interpreter für ctrlX OS 02VRS Anwendungsbeschreibung ↗ Link zur Web-Dokumentation Bestellinformationen: • DOK-XCORE*-GCO***V02**-APRS-DE-P • R911421630
24	Key Value Database App - Verwaltung von Daten im Data Layer 02VRS Anwendungsbeschreibung ↗ Link zur Web-Dokumentation Bestellinformationen: • DOK-XCORE*-KVD*****V02-APRS-DE-P • R911421634
25	OPC UA Pub/Sub App - OPC UA Pub/Sub für ctrlX OS 02VRS Anwendungsbeschreibung ↗ Link zur Web-Dokumentation Bestellinformationen: • DOK-XCORE*-UAP***V02**-APRS-DE-P • R911421601

Nr.	Dokumentation
26	Modbus TCP App - Modbus TCP-Kommunikation für ctrlX OS 02VRS Anwendungsbeschreibung ↗ Link zur Web-Dokumentation Bestellinformationen: • DOK-XCORE*-MOD*TCP*V02-APRS-DE-P • R911421620
27	Service Indicator App -Service Indicator für ctrlX OS 02VRS Anwendungsbeschreibung ↗ Link zur Web-Dokumentation Bestellinformationen: • DOK-XCORE*-SIN*****V02-APRS-DE-P • R911421626
28	Model Connect App - Target für modellbasierte Entwicklung und Simulation für ctrlX OS 02VRS Anwendungsbeschreibung ↗ Link zur Web-Dokumentation Bestellinformationen: • DOK-XCORE*-MOC***V02**-APRS-DE-P • R911421630
29	System Overview App - Systemtopologie und Systeminformationen 02VRS Anwendungsbeschreibung ↗ Link zur Web-Dokumentation Bestellinformationen: • DOK-XCORE*-SOV***V02**-APRS-DE-P • R911424408
30	Cam Designer - Kurvenscheiben der ctrlX MOTION konfigurieren 02VRS Anwendungsbeschreibung ↗ Link zur Web-Dokumentation Bestellinformationen: • DOK-XWORKS-CAM***V02**-APRS-DE-P • R911424389

15 Service und Support

Für Ihre schnelle und optimale Unterstützung verfügen wir über ein dichtes weltweites Servicenetz. Unsere Experten stehen Ihnen mit Rat und Tat zur Seite. Sie erreichen uns täglich **rund um die Uhr – auch an Wochenenden und Feiertagen**.

Service Deutschland

Unser technologieorientiertes Competence Center in Lohr deckt alle Belange rund um den Service für elektrische Antriebe und Steuerungen ab.

Sie erreichen unsere **Service-Hotline** und unseren **Service-Helpdesk** unter:

Telefon: **+49 9352 40 5060**

Fax: **+49 9352 18 4941**

E-Mail: [↗ service.svc@boschrexroth.de](mailto:service.svc@boschrexroth.de)

Internet: [↗ http://www.boschrexroth.com](http://www.boschrexroth.com)

Auf unseren Internetseiten finden Sie ergänzende Hinweise zu Service, Reparatur (z. B. Anlieferadressen) und Training.

Service weltweit

Außerhalb Deutschlands nehmen Sie bitte zuerst Kontakt mit Ihrem Ansprechpartner auf. Die Hotline-Rufnummern entnehmen Sie bitte den Vertriebsadressen im Internet.

Vorbereitung der Informationen

Wir können Ihnen schnell und effizient helfen, wenn Sie folgende Informationen bereithalten:

- Eine detaillierte Beschreibung der Störung und der Umstände
- Angaben auf dem Typenschild der betreffenden Produkte, insbesondere Typenschlüssel und Seriennummern
- Ihre Kontaktdaten (Telefon-, Faxnummer und E-Mail-Adresse)

16 Index

B

Bestimmungsgemäßer Gebrauch

Anwendungsbereiche	6
Einleitung	5
Einsatzfälle	6

C

ctrlX AUTOMATION

Weiterführende Dokumentationen	49
--------------------------------------	----

F

Fenster

PLC	45
-----------	----

H

Header

PLC	46
-----------	----

Helpdesk	55
----------------	----

Hotline	55
---------------	----

L

Lizenzierung der Multicore-Funktionalität	15, 16
---	--------

M

Multicore Einführung	14
----------------------------	----

N

Nicht bestimmungsgemäßer Gebrauch	7
---	---

Folgen, Haftungsausschluss	6
----------------------------------	---

P

PLC App

Data Layer-Anbindung	13
Flexible Konfiguration des Speichers für remanente SPS-Daten	22
Grundlagen	7, 10
Registerkarte "Anwendungen"	47
Registerkarte "Boot-Anwendungen"	48
Registerkarte "Projektinformationen"	48
Seitennavigation	44
Zielsysteme	12

S

Service-Hotline	55
-----------------------	----

Sicherheitshinweise	7
---------------------------	---

SPS Laufzeitsystem

Anbindung an das ctrlX OS Systemverhalten	23
--	----

SPS-Task

Extern-ereignisgesteuerte SPS-Tasks	41
Frei konfigurierbare, externe Ereignisquellen	42

Support	55
---------------	----

Bosch Rexroth AG
Bgm.-Dr.-Nebel-Str. 2
97816 Lohr a.Main
Germany
Tel. +49 9352 18 0
Fax +49 9352 18 8400
www.boschrexroth.com/electrics



R911423400 03